CARAZONE: Federated Intrusion Detection for Timing Attacks in Automotive Zonal TSN Architectures

Project Team

Adeena Zaib 23I-0674
Khaula Bilqees 23I-0574

Session 2023-2027

Supervised by

Dr. Muhammad Rizwan

Co-Supervised by

Dr. Hina Ayaz



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Contents

Students' Submission i

Anti-Plagiarism Statement ii

Authors' Declaration iii

Abstract iv

Acknowledgments v

Executive Summary vi

List of Abbreviations vii

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 2
1.5 Project Objectives . . . 2
1.6 Project Scope . . . 3
1.7 Project Modules . . . 5
1.8 Stakeholders and User Classes . . . 5
1.9 Success Criteria . . . 6
1.10 Constraints and Assumptions . . . 7
1.11 Ethical, Legal and DEI Considerations . . . 8

2 Research and Existing Solutions 10

2.1 Research Method / Approach . . . 10
2.2 Existing Solutions / Products . . . 10
2.3 Comparison of Existing Solutions . . . 12
2.4 Limitations and Gaps . . . 12
2.5 Proposed Solution . . . 13
2.6 Complex Computing Problem . . . 14

2.7 Technical Challenges . . . 15
2.8 Research Findings and Design Decisions . . . 15

3 Proposal Feedback and Iteration Plan 18
3.1 Proposal Feedback Summary . . . 18
3.2 Proposal Action Register . . . 18
3.3 Iteration Strategy . . . 21
3.4 Iteration 1 Plan . . . 21
3.5 Iteration 2 Plan . . . 22
3.6 Work Breakdown and Responsibilities . . . 22
3.7 Progress Against Plan . . . 23
3.8 Delays, Deviations and Recovery Plan . . . 23

4 Software Requirements 25
4.1 User Interactions / Use Cases . . . 25
4.2 Use Case Diagram . . . 25
4.3 Detailed Use Cases . . . 25
4.4 Functional Requirements . . . 27
4.4.1 Module 1: Simulation and Attack Injection . . . 27
4.4.2 Module 2: Feature Extraction . . . 28
4.4.3 Module 3: Detection and Federation . . . 28
4.4.4 Module 4: Alerting and Evaluation . . . 29
4.5 Non-Functional Requirements . . . 29
4.5.1 Reliability . . . 29
4.5.2 Usability . . . 29
4.5.3 Performance . . . 29
4.5.4 Security . . . 30
4.6 Acceptance Criteria . . . 30
4.7 Requirements Traceability . . . 30

5 System Design 32
5.1 System Architecture . . . 32
5.2 Detection Methodology and Deployment View . . . 33
5.3 Architectural Components . . . 34
5.4 Interfaces and Dependencies . . . 35
5.5 Architecture Decisions and Rationale . . . 36
5.6 Reference Topology Design . . . 38
5.6.1 Basis of the design . . . 39
5.6.2 Zone partitioning . . . 39
5.6.3 Interconnect . . . 40
5.6.4 Time synchronisation . . . 40

5.6.5 Generation and reproducibility . . . 41
5.6.6 Validation . . . 41
5.6.7 Expected heterogeneity . . . 42
5.6.8 Limitations . . . 43
5.7 Design Evolution . . . 43
5.8 Domain Model . . . 44
5.9 Data Design . . . 45
5.10 Process / Behavior Design . . . 47

6 Iteration 1: Design and Implementation 50
6.1 Iteration Objectives . . . 50
6.2 Selected Use Cases / Features . . . 50
6.3 Design . . . 50
6.4 Implementation . . . 51
6.5 Completed Features / Module . . . 51
6.6 Technical Implementation Details . . . 52
6.7 Algorithms and Technical Decisions . . . 53
6.8 Tools, Frameworks and Technologies . . . 54
6.9 Demonstration Evidence . . . 57
6.10 Attack Simulation Results on the Ring Topology . . . 57
6.10.1 Simulation pipeline and scenarios . . . 57
6.10.2 Benign baseline . . . 58
6.10.3 Constant drift attack . . . 58
6.10.4 Asymmetric delay attacks . . . 59
6.10.5 FRER attacks . . . 60
6.10.6 Detection signals identified . . . 61
6.10.7 Limitations of these results . . . 61
6.11 Iteration Outcome . . . 62

7 Testing and Early Validation 63
7.1 Testing Strategy . . . 63
7.2 Test Levels and Techniques . . . 63
7.3 Test Cases . . . 63
7.4 Test Results . . . 64
7.5 Acceptance Testing / Validation . . . 65
7.6 Edge and Failure Cases . . . 67
7.7 Defects and Corrective Actions . . . 70
7.8 Current Validation Status . . . 71

8 Project Management and Development Evidence 72
8.1 Work Completed . . . 72

8.2 Work Remaining . . . 72
8.3 Individual Responsibilities . . . 72
8.4 Contribution of Each Team Member . . . 73
8.5 Repository and Version Control Evidence . . . 73
8.6 Milestones and Commit Evidence . . . 74
8.7 Tools and Tool-Assisted Development . . . 75
8.8 External Components and Dependencies . . . 75
8.9 Risks and Mitigation . . . 76
8.10 Security and Privacy Considerations . . . 77
8.11 Current Limitations . . . 77

9 Remaining FYP Plan **79**
9.1 Remaining Scope . . . 79
9.2 Remaining Modules and Features . . . 79
9.3 Remaining Iterations . . . 79
9.4 Implementation Plan . . . 80
9.5 Testing and Validation Plan . . . 80
9.6 Deployment Plan . . . 81
9.7 Tentative Timeline . . . 82
9.8 Expected Final Deliverables . . . 83

10 Conclusion **84**

A Proposal Action Register: Supporting Evidence **86**

B Detailed Requirements **87**

C Detailed Test Cases **88**

D Repository and Contribution Evidence **89**

E Additional Design Diagrams **91**

F Meeting and Progress Evidence **92**

List of Figures

4.1 Use case diagram. Human actors are shown left, system actors right. . . . 26

5.1 Layered system architecture with interface contracts. 32
5.2 Detection methodology and federated deployment 33
5.3 Five-zone ring reference topology 41
5.4 Domain model 45
5.5 Level-1 data flow diagram. 47
5.6 Feature extraction activity 48
5.7 Detector state machine 48

6.1 Implemented four-zone tree topology 51
6.2 Clock offset, benign run 55
6.3 Clock offset, delay attack on zone 0 56

8.1 GitHub Actions run history 74

List of Tables

2.1 Reviewed solutions, results and limitations . . . 11
2.2 Coverage of the three required dimensions . . . 12

3.1 Proposal action register . . . 19
3.2 Work breakdown by member . . . 22
3.3 Planned milestones versus actual progress . . . 23

4.4 Requirements traceability matrix . . . 31

5.3 Architecture decision record . . . 36
5.4 Zone composition of the reference topology . . . 39
5.6 `features.csv` column contract . . . 45

6.2 Ring scenario catalogue. All drift and FRER configurations run for two simulated seconds. . . . 58
6.3 Constant drift on `lidarRR`: the servo's correction records the attack that the corrected clock hides. . . . 59
6.4 Asymmetric delay on `swFL`: offset shift at the victim, a downstream node and a control node. . . . 59
6.5 FRER signals under the two attacks. Each attack is visible in a different counter. . . . 60
6.6 Candidate detection features identified by the ring attack simulations. . . . 61

7.1 Test cases executed at the mid-report stage . . . 64
7.2 Label distribution by zone . . . 64
7.3 Feature ablation, macro-F1 averaged over five run-level splits . . . 66
7.4 Absolute sibling offset residual by population . . . 68
7.5 Defect register . . . 70

8.1 Milestones against the commits that completed them . . . 74
8.2 Tool and AI-assistance disclosure . . . 75
8.4 Risk register . . . 76

9.2 Tentative FYP-2 timeline . . . 82

F.1 Supervisor meetings and their outcomes . . . 92

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *CARAZONE: Federated Intrusion Detection for Timing Attacks in Automotive Zonal TSN Architectures*.

Student 1: Adeena Zaib Signature: _____

Student 2: Khaula Bilqees Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable.

Abstract

Automotive zonal electrical/electronic architectures place safety-critical control traffic, sensor streams and infotainment on a single Ethernet backbone, and rely on Time-Sensitive Networking (TSN) to keep that traffic deterministic. Determinism depends on the generalised Precision Time Protocol (gPTP, IEEE 802.1AS), which was standardised without security controls. An attacker who manipulates synchronisation can therefore degrade timing guarantees without altering packet content or traffic volume, which places the attack outside the detection surface of conventional content- and volume-based intrusion detection.

This project develops a federated intrusion detection system that operates on TSN protocol-native signals rather than generic traffic statistics. Zone controllers act as federated clients, each extracting timing features from locally observed gPTP state and training a shared model architecture; a central vehicle compute unit aggregates model parameters without receiving raw feature records.

At the mid-report stage, a four-zone automotive TSN topology has been implemented in OMNeT++/INET with gPTP synchronisation, and three attack configurations have been instrumented. Fifteen seeded simulation runs (three benign and twelve delay-attack runs covering four target zones at three seeds each) have produced 225 recorded vectors, converted by a Python extraction pipeline into 4,740 labelled feature windows. Delay-attack injection has been validated against the expected theoretical signature. A cross-zone residual feature has been added and a centralized detector trained, establishing the project's central premise ahead of schedule: features computed within a single zone detect the attack no better than a majority-class predictor (macro-F1 0.48–0.54 against a floor of 0.47), whereas two cross-zone residual features reach 0.95–0.98 macro-F1 with detection two synchronisation intervals after onset. Two negative results are reported: oscillator drift manipulation is not realisable as a runtime attack under the simulation framework, and no fixed threshold, neither a maximum nor a percentile, separates attack from benign behaviour, because the synchronisation convergence transient produces residuals 2.7 times larger than the attack does. Remaining work for FYP-2 comprises zone heterogeneity, the federated training pipeline, and the comparative evaluation.

Acknowledgments

The project team acknowledges the guidance and support received from the project supervisor, faculty members, and other contributors.

Executive Summary

Problem. Modern automotive systems increasingly use a shared TSN Ethernet backbone to connect vehicle components that were previously separated into individual domains. As a result, isolation that was once provided by separate physical networks now relies on mechanisms such as time-aware scheduling, which depends on a common time base distributed by gPTP. However, gPTP does not provide authentication, leaving it vulnerable to attacks such as asymmetric delay injection. Cryptographic measures alone cannot prevent these attacks, as delaying a message can be as effective as forging one. Consequently, intrusion detection systems based on payload inspection or traffic statistics may fail to detect these timing-related attacks.

Gap. Existing work divides into three groups, none of which covers this intersection: industrial federated TSN-IDS work that is not automotive or zonal; automotive IDS work that targets the CAN bus and therefore has no gate schedule or synchronisation tree to observe; and TSN security research that characterises attack impact without federated, zone-level operational detection.

Approach. A centralized cross-silo horizontal federated learning design in which each zone controller is a client. Clients extract windowed timing features from gPTP state, train a shared compact model locally, and exchange only parameters with the central vehicle compute unit. The contribution is the use of protocol-native, schedule-derived signals as the detection feature space, evaluated against a generic-traffic-feature control arm and against centralized and local-only training baselines.

Status at mid-report. The simulation environment, attack injection harness and feature extraction pipeline are complete and reproducible across seeded runs. A centralized detector has been trained and the feature ablation establishing success criterion SC-2 has been completed. The federated layer is designed but not yet implemented. One attack class is validated, one is documented as unrealisable under the current framework, and one is instrumented but not yet extracted.

Principal risk. All zones currently carry identical traffic, which would make the federated comparison ineffective; differentiated per-zone profiles are scheduled first in Iteration 04. A secondary concern is that the injected attack is large relative to the benign steady-state envelope, so an attack-intensity sweep is required to establish where detection degrades.

List of Abbreviations

AVTP	Audio Video Transport Protocol
BMCA	Best Master Clock Algorithm
CVC	Central Vehicle Compute
E/E	Electrical/Electronic (architecture)
FedAvg	Federated Averaging
FL	Federated Learning
FR	Functional Requirement
FRER	Frame Replication and Elimination for Reliability (IEEE 802.1CB)
GCL	Gate Control List
GM	Grandmaster (clock)
gPTP	generalised Precision Time Protocol (IEEE 802.1AS)
HFL	Horizontal Federated Learning
IDS	Intrusion Detection System
MLP	Multilayer Perceptron
NFR	Non-Functional Requirement
TAS	Time-Aware Shaper (IEEE 802.1Qbv)
TSN	Time-Sensitive Networking
TT	Time-Triggered (traffic class)
vSOC	Vehicle Security Operations Centre
ZC	Zone Controller

Chapter 1

Introduction and Project Vision

1.1 Project Overview

CARAZONE is a simulation-based federated intrusion detection system for the Time-Sensitive Networking (TSN) Ethernet backbone of an automotive zonal electrical/electronic architecture. The system observes TSN protocol-native signals at each zone controller, extracts timing features over fixed observation windows, trains a shared detection model across zones using federated learning without centralising raw feature records, and emits attributed alerts identifying the attack class and the affected zone.

The delivered artefact is a laboratory validation and research tool, not a production vehicle component. It comprises a configurable zonal TSN simulation, an attack injection harness producing ground-truth labels, a feature extraction pipeline, a federated training and evaluation workflow, and an alert presentation surface.

1.2 Problem Statement

No existing work federates intrusion detection built on TSN's own protocol-native signals for automotive zonal Ethernet backbones, a topology with heterogeneous zone-level data that current federated TSN-IDS work (industrial) and automotive IDS work (CAN bus) both miss.

Concretely: in a zonal architecture the isolation between safety-critical and non-critical traffic is enforced logically by time-aware scheduling rather than physically by separate wiring. That scheduling is meaningful only because every node agrees on the current time. An adversary who corrupts the shared time base therefore degrades timing guarantees across the backbone without injecting malformed frames, without raising traffic volume, and without altering any payload. Detection approaches that observe payload content or

flow statistics have no visibility of this failure mode.

1.3 Problem Context and Motivation

Automotive E/E architecture is migrating from roughly 70–100 distributed ECUs on segmented buses toward a small number of zone controllers linked by a high-bandwidth Ethernet backbone. TSN provides the determinism that the displaced buses supplied natively. Two TSN mechanisms are central: IEEE 802.1AS (gPTP) distributes a common time base, and IEEE 802.1Qbv (TAS) uses that time base to gate traffic classes into scheduled windows.

Neither mechanism was standardised with security controls. For gPTP specifically, cryptographic authentication does not close the full threat surface: an asymmetric delay attack manipulates the *timing* of authentic messages, corrupting the peer-delay computation on which offset correction depends. Authentication confirms that a message is genuine; it cannot establish that the message arrived when it should have.

The consequence is not immediate catastrophic failure. End-to-end protection mechanisms at the application layer detect missed updates and degrade to fail-safe states. The realistic impact is stealthy availability denial: forced degradation, limp-home transitions and driver warnings at a time of the attacker's choosing. This is a detection problem rather than a prevention problem, and it is the problem this project addresses.

1.4 Project Vision

To demonstrate that intrusion detection for automotive zonal TSN backbones is more effective when built on protocol-native timing signals than on generic traffic statistics, and that such detection can be trained across heterogeneous zones using federated learning without centralising raw per-zone observations.

1.5 Project Objectives

1. **O1.** Implement a configurable multi-zone automotive TSN topology in OMNeT++/INET with gPTP synchronisation and reproducible seeded execution.
2. **O2.** Implement an attack injection harness for TSN timing attacks that produces ground-truth labels with zone, time-window and attack-class attribution.

3. **O3.** Implement a passive feature extraction pipeline converting raw simulation output into labelled, windowed, protocol-native feature vectors.
4. **O4.** Train and evaluate a detection model on those features, and demonstrate by controlled comparison whether protocol-native features outperform generic traffic features on the same scenarios.
5. **O5.** Implement federated training across per-zone clients and compare it against centralized-pooled and local-only training baselines.
6. **O6.** Produce attributed alerts carrying attack class, affected zone, time window and confidence, and present them through an operator-facing surface.

Objectives O1–O3 are complete at the mid-report stage. O4–O6 are scheduled for Iterations 03 and 04.

1.6 Project Scope

Scope statement

The project targets the TSN Ethernet backbone connecting zone controllers to central compute in automotive zonal E/E architectures, focusing specifically on attacks against TSN features. The project trains a detection model across simulated zone/platform clients standing in for heterogeneous vehicle architectures (zonal topologies), using federated learning. Evaluation compares this federated approach against a centralized version of the same detector and against existing non-federated detection work, using synthetic attack datasets generated via a TSN simulation framework.

Included and excluded functionality

Included (committed)	Excluded
Multi-zone TSN topology, simulated in OMNeT++/INET with gPTP synchronisation	Hardware testbed or physical vehicle validation
Attacks against gPTP synchronisation (delay injection, sync starvation) and against FRER (IEEE 802.1CB) stream redundancy	Link-layer attacks outside the TSN control and redundancy planes

Included (committed)	Excluded
Passive extraction of protocol-native timing features over fixed windows	Active probing, packet modification, or in-path interference
Supervised multiclass detection (benign + attack classes)	Unsupervised/one-class detection as the primary mechanism
Federated training (FedAvg) with centralized and local-only baselines	Secure aggregation, differential privacy, model-poisoning defence
Zone-level attack attribution	ECU-level or port-level localisation
Operator-facing alert presentation surface	Production vehicle integration or in-cabin driver interface

Scope items in progress, and deferred scope

FRER-based detection (in scope; simulated on the ring, feature extraction pending). Frame Replication and Elimination for Reliability is committed scope. It is not exercised in the Iteration 1 dataset because the four-zone tree topology gives each zone a single path to the backbone, and FRER requires disjoint redundant paths between talker and listener. A five-zone ring reference topology removes that dependency (Section 5.6); its automated check confirms that cutting any single ring or uplink link partitions nothing, which is the machine-checkable form of the disjoint-path requirement. IEEE 802.1CB has since been configured on the ring for a protected brake-control stream, replication and duplicate elimination have been confirmed, and two stream-redundancy attacks, a path cut and a same-port flood, have been simulated and analysed, each leaving a distinct signature (Section 6.10). The remaining work is bounded and specified: adding the FRER feature family to the extraction pipeline (inter-member arrival skew, sequence-number deviation between redundant members, and inter-arrival cadence per member), and implementing a protocol-level stream-redundancy attack. The attack model is stream-redundancy manipulation: selective suppression or delay of one member of a replicated stream, which degrades the reliability guarantee while every frame remains individually authentic. This family is attractive for the project's central claim because its features are structurally independent of the gPTP offset residuals, so it tests whether the cross-zone residual principle generalises beyond the synchronisation plane. Scheduling and the exit criterion are given in Section 9.3.

Schedule-window (802.1Qbv) violation detection (deferred). TAS gate scheduling is not yet configured in the topology, so gate-conformance features cannot be computed. This is retained as a planned extension because it supplies a second, independent protocol-

native feature family and a direct physical consequence measure (deadline misses).

The 802.1Qbv deferral follows the scope-protection rule established in the project proposal, which identifies scope expansion through additional attack classes as a high-impact risk and names extension features as the first items to drop. FRER is distinguished from that rule because it is a committed objective with a bounded dependency rather than an opportunistic addition.

1.7 Project Modules

#	Module	Responsibility
M1	Zonal Topology Simulation	Constructs the multi-zone TSN network model: grandmaster, backbone switch, zone controllers, endpoint ECUs, central vehicle compute; gPTP synchronisation; seeded reproducible execution.
M2	Attack Injection & Dataset	Injects TSN protocol-native attacks with configurable start time, intensity and target; emits ground-truth labels with zone, window and class attribution.
M3	Feature Extraction Layer	Passively converts raw per-node simulation output into fixed-size windowed feature vectors using protocol-native signals rather than generic traffic statistics.
M4	Federated Detection Model	Trains a shared detection model across per-zone clients; produces the global model distributed back to zones for inference.
M5	Zone Participation & Aggregation	Governs how zones with differing traffic profiles contribute to the shared federated model.
M6	Alerting & Presentation	Packages classifications into attributed alerts and presents them through an operator-facing incident queue.

Modules M1–M3 are implemented. M4–M6 are designed and scheduled.

1.8 Stakeholders and User Classes

Stakeholder	Need	Evidence required
Automotive network security / validation engineer	Determine whether timing attacks are detectable in a candidate zonal TSN configuration	Attributed alerts with class, zone, time window, confidence; reproducible scenarios
Vehicle Security Operations Centre (vSOC) analyst	Triage incoming incidents across a fleet and distinguish fault from campaign	Incident queue with severity, zone, attack class, confidence and status
TSN validation engineer	Confirm that monitoring does not disturb protected traffic	Matched monitoring-enabled vs. monitoring-disabled timing measurements
Project supervisor / evaluator	Judge whether the approach is technically justified	Baseline comparison, reproducible experiments, documented failure analysis
Future researcher	Reproduce or extend the work	Versioned configuration, seeds, raw outputs, run instructions

Operating context. The intended deployment position of the detector is a zone controller within the vehicle, with aggregation on the central vehicle compute unit. In this project the entire system is exercised in simulation on a development workstation; no claim of in-vehicle deployment is made.

1.9 Success Criteria

Success is assessed against measurable criteria rather than a single aggregate accuracy figure. Class imbalance in the generated data (see Section 7.4) makes accuracy uninformative, so it is excluded as a headline measure.

ID	Criterion	Measure and target
SC-1	Attack effects are measurable and repeatable	Injected attacks produce a timing signature distinguishable from benign behaviour across $\geq$3 seeds, with ground-truth labels. **Met** for delay injection.

ID	Criterion	Measure and target
SC-2	Cross-zone features outperform within-zone features	Identical model trained on (a) cross-zone residual and (b) within-zone statistics over identical run-level splits; macro-F1 of (a) exceeds (b) by a margin that holds across seeds and across models. **Met**: $+0.43$ to $+0.49$ macro-F1 over three models (Section 7.5).
SC-3	Detection is operationally timely	Detection latency, reported in synchronisation intervals from attack onset. **Partially met**: two intervals under a three-window confirmation rule (Section 7.5). Comparison against a schedule margin awaits time-aware shaping.
SC-4	Federated training is justified	FedAvg macro-F1 exceeds local-only training, and the gap to centralized-pooled training is quantified. A null result is reported as a finding, not concealed.
SC-5	Attribution is correct	Zone attribution accuracy (top-1) reported separately from detection performance.
SC-6	Alert volume is operationally acceptable	False alarms per vehicle-hour reported at the chosen operating point, in addition to false-positive rate.
SC-7	Results are reproducible	A documented configuration and seed regenerate the same scenario structure and ground-truth labels on a clean environment.

1.10 Constraints and Assumptions

Constraints

- **C1.** Evaluation is simulation-based. No hardware TSN testbed or vehicle platform is available, so all traffic and all attacks are synthetic.
- **C2.** Observable signals are limited to those the simulation framework exposes. Features depending on internal servo state must be instrumented explicitly, and availability is verified per feature before commitment.
- **C3.** Development and simulation run on a single workstation; the federated setting

is simulated (clients as processes), not physically distributed.

- **C4.** Two-member team over two semesters, which bounds the number of attack classes and model variants that can be implemented and defended.

Assumptions

- **A1. Static network environment.** The topology, the gPTP domain configuration and (when introduced) the gate schedule are statically configured and do not reconfigure at runtime. This assumption is load-bearing: the detection approach uses deviation from a known, configured expectation, which is available *a priori* only in a static environment. In a dynamically reconfigured network, legitimate reconfiguration events would present as anomalies and would require explicit handling. Dynamic reconfiguration is out of scope and recorded as a limitation.
- **A2. Static (one-shot) training.** Federated training occurs once, offline, before deployment. After the global model is distributed, clients perform inference only; no online retraining occurs. Continuous federated learning would require solving live ground-truth labelling in deployment, which is out of scope.
- **A3.** The grandmaster is statically assigned and trusted. Attacks originate from a compromised on-path node within a zone, not from the time source itself. Grandmaster-level compromise is addressed as a scope boundary in Section 2.7.
- **A4.** The aggregation server (central vehicle compute) is trusted. Secure aggregation and model-poisoning resistance are out of scope.
- **A5.** Monitoring is passive and sits outside the forwarding path.

1.11 Ethical, Legal and DEI Considerations

Ethical. The project uses only synthetic, simulated data. No real vehicle data, no OEM data and no personal data are collected or processed. No human subjects are involved. Attack implementations target a simulated network only; no capability is developed against any real or deployed system.

Legal. No proprietary OEM communication matrices or topology data are used. OMNeT++ and INET are used under their respective open-source licences, which are recorded in the dependency register (Section 8.8). Published datasets, if incorporated for validation, will be used under their stated academic-use terms with required citation.

DEI and bias. Synthetic topologies are constructed from published architectural patterns rather than any specific manufacturer's proprietary design, which avoids embedding a single vendor's assumptions into the evaluation. The project produces no output that classifies or affects individuals.

Dual-use consideration. The attack implementations are confined to the simulation environment and are described at the level of effect rather than as a reusable exploit toolkit.

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

The literature was investigated in three directions corresponding to the three bodies of work the project sits between: (i) federated learning for intrusion detection [1–11], (ii) automotive in-vehicle network security [12–19], and (iii) TSN protocol security, specifically gPTP [20–30]. Sources were drawn from IEEE Xplore, ACM Digital Library, ScienceDirect and arXiv, together with the relevant IEEE 802.1 standards documents.

Each source was recorded against four fields (methodology, results, limitations, and the design decision it influenced) so that the review produces design consequences rather than a summary. Candidate sources were retained only where they addressed at least one of: TSN protocol-native detection signals, federated aggregation under non-IID clients, or automotive zonal architecture.

In addition, a comparison was performed against the departmental FYP repository (Fall 2019 – Fall 2025) to establish that the project does not duplicate prior student work. The result of that comparison is reported in Section 2.3.

2.2 Existing Solutions / Products

Table 2.1: Reviewed solutions, results and limitations

Source	Methodology	Results	Limitations
SAFID [1]	Selective federated learning, TDNet, domain adaptation, post-training quantization	99.75% accuracy on CICIDS2017; 95.75% on Edge-IIoTset; 6.5× inference-throughput improvement	Evaluated on generic IDS datasets rather than automotive zonal TSN traces; industrial segmented networks, not vehicle zonal
Yücel et al. [25]	Physical avionics TSN testbed; unsupervised anomaly detection using Local Outlier Factor on gPTP timing features	Detection at 500 ns and 1000 ns offsets; F1 remained above 0.95 even for 25 ns and 50 ns attacks	Avionics testbed, not automotive zonal TSN; no federated learning; single synchronisation domain; unsupervised framing driven by label scarcity that does not apply to a simulation-generated dataset
TSNZeek [21]	Zeek extension with Spicy packet parsing and rule-based detection for TSN protocols	≈5% CPU overhead; 25% CPU at 50 Mbit/s, 45% at 250 Mbit/s; up to 2 ms processing lag; detected all seven tested attacks	Does not address gPTP timing attacks, federated learning, or automotive zonal deployment; rules are stateful and centralized
Sorescu et al. [18]	Model-based TARA using STRIDE and ISO/SAE 21434 assumptions	151 distributed, 259 domain-based and 270 zonal threat rows	Threat counts are model outputs, not measured attack probabilities or IDS results
Adil et al. [9]	Federated collaborative learning, NTTE traffic encoding, TCNnet, INT8 quantization, TIMM safety modelling	Up to 99.98% accuracy; model size reduced from 2.81 MB to 0.23 MB; collision reduction up to 97.9%	Real-time vehicle integration, rare attacks and protocol-specific anomalies remain unresolved

2.3 Comparison of Existing Solutions

Three dimensions are jointly required by this project: federated training, an automotive zonal setting, and detection built on TSN protocol-native signals. Table 2.2 evaluates the reviewed literature and the closest prior departmental FYPs against those dimensions.

Table 2.2: Coverage of the three required dimensions

Work	**FL**	**Automotive / zonal**	**TSN-native signals**
SAFID [1]	Yes	No	No
Yücel et al. [25]	No	No (avionics)	Yes
TSNZeek [21]	No	No (industrial)	Partial (rule-based)
Adil et al. [9]	Yes	Yes (CAN/embedded)	No
F25-151-R-DIDS-IoV (FYP)	Yes	Yes (IoV, not zonal)	No
F25-209-R-BastionFed (FYP)	Yes	No	No
F25-378-D-AlertEye (FYP)	Yes	No	No
F25-178-R-SafeCharge (FYP)	No	EV charging, off-vehicle	No
F25-168-D-NeuroDrive (FYP)	No	Yes (OBD-2 bus)	No
CARAZONE (this work)	**Yes**	**Yes**	**Yes**

No reviewed work covers all three dimensions. One prior departmental project (DIDS-IoV) covers two. Across the departmental repository from Fall 2019 to Fall 2025, no project addresses TSN, gPTP, FRER, 802.1 deterministic networking or automotive Ethernet; the single automotive in-vehicle network security project targets the CAN bus.

2.4 Limitations and Gaps

The reviewed work exhibits four limitations relevant to this project:

1. **Feature space.** Enterprise, IoT and industrial IDS work operates on protocol-agnostic statistical traffic features, because those networks offer no configured schedule or synchronisation structure to compare against. In a deterministic network, the expected value of a timing observation is known in advance from configuration. No reviewed work exploits this asymmetry in an automotive zonal setting.

2. **Domain transfer.** Work that does use gPTP timing signals is situated in avionics, with a single synchronisation domain and a different traffic profile from a vehicle zonal backbone.

3. **Federated evaluation realism.** Federated IDS work commonly evaluates on artificial IID partitions. Zone-level heterogeneity in a vehicle is structural, since a perception zone and a body zone differ in traffic density, stream composition and criticality, and is not represented by a random split.

4. **Operational framing.** Several works report classification quality without detection latency, alert volume at realistic base rates, or the effect of monitoring on the protected traffic it observes.

Gap statement. No existing work provides federated, zone-level intrusion detection for automotive TSN backbones using protocol-native timing signals, with evaluation that reports detection quality, attribution, latency and federated behaviour under structurally heterogeneous clients.

2.5 Proposed Solution

CARAZONE addresses the gap with a centralized cross-silo horizontal federated learning architecture in which each zone controller is a client. Each client observes local gPTP state passively, computes windowed protocol-native features, and trains a shared compact model architecture. The central vehicle compute unit aggregates parameters using FedAvg and redistributes the global model. Clients then perform inference locally and emit attributed alerts.

Two design choices follow directly from the gap:

Protocol-native features. Features are computed as residuals against values that TSN configuration makes known in advance: deviation of a node's offset from its siblings in the synchronisation tree, deviation of sync message arrival from the configured interval, and the variance of measured link delay. These are not available to a generic traffic-feature detector, and the controlled comparison against such a detector is the project's primary experiment.

Federation across zones rather than vehicles. Clients are zones within one vehicle, so the aggregation server is the vehicle's own central compute unit. The motivation is not inter-organisational privacy but bandwidth: streaming full timing traces from every zone to a central analyser is unbounded best-effort traffic competing with the scheduled streams the system protects, whereas periodic exchange of fixed-size model parameters is a bounded, schedulable traffic pattern.

2.6 Complex Computing Problem

The project addresses a complex computing problem under the Seoul Accord characterisation. Complexity does not arise from combining TSN, machine learning and federated learning; it arises from the interaction between timing guarantees, low-observable attacks, heterogeneous distributed training and operational constraints.

Characteristic	**Manifestation in this project**
Conflicting requirements	Detection sensitivity competes with false-alarm volume, inference cost on a resource-constrained zone controller, federated communication overhead, and the requirement that monitoring must not perturb the protected traffic it observes.
No obvious solution	A stealthy delay attack can be constructed to remain inside the envelope of benign oscillator drift, so no fixed threshold separates the two; and federated aggregation may degrade rather than improve performance under genuinely non-IID clients.
Depth of knowledge	Requires IEEE 802.1AS synchronisation mechanics, time-aware scheduling, discrete-event network simulation, timing-feature design, and federated optimisation.
Beyond standard practice	Neither IEEE 802.1AS nor IEEE 802.1Qbv defines a security or detection layer. A routine IDS or a routine FL implementation provides neither timing-aware attribution nor evaluation of monitoring interference.
Significant consequences	Missed detection permits forced degradation of safety-relevant functions; excessive false alarms make the system operationally unusable at fleet scale.
Interdependence	Synchronisation behaviour, feature extraction, local training, aggregation strategy and deployment inference cost are mutually constraining; a change in window size propagates to detection latency, model size and communication volume.

2.7 Technical Challenges

1. **Observability of internal protocol state.** Several informative features derive from the synchronisation servo's internal state rather than from any field visible in a packet capture. Whether the simulation framework exposes these, or whether explicit instrumentation is required, must be verified per feature before the feature is committed.

2. **Benign/attack separability.** Benign oscillator behaviour includes thermal and ageing drift whose magnitude can exceed that of a deliberately stealthy attack. Detection must therefore rely on structural rather than magnitude-based discrimination.

3. **Grandmaster-level blind spot.** Detection based on comparison between siblings in the synchronisation tree is, by construction, insensitive to a perturbation that moves the entire tree together. This was confirmed empirically (Section 7.6) and is recorded as a scope boundary: the system addresses link-local timing integrity, not time-source integrity.

4. **Structural client heterogeneity.** For the federated comparison to be meaningful, zones must differ in traffic profile. A symmetric topology makes federated, centralized and local-only training converge to equivalent models and renders the comparison null by construction.

5. **Class imbalance and evaluation validity.** Attack windows are a small minority of all windows. Accuracy is therefore uninformative, and train/test splitting must occur at scenario and seed granularity rather than at window granularity to prevent leakage between adjacent correlated windows.

2.8 Research Findings and Design Decisions

Finding	Source	Design decision
Automotive TSN can be represented with domains, gateways, periodic streams and scheduling in a discrete-event simulator	Zhou et al. [19]	Use OMNeT++/INET rather than building a simulator from scratch; keep the topology small and seeded

Finding	Source	Design decision
Clock offset, port state and synchronisation message behaviour expose automotive Ethernet synchronisation attacks	Fotouhi et al. [28]	Commit offset, offset trend and synchronisation-interval features as the core feature family
Some PTP attacks are stealthy and consume very little bandwidth	Berardi et al. [29]	Do not rely on packet-volume features; make the generic-traffic-feature detector the control arm rather than the baseline to beat on volume
gPTP timing features support detection at small offset magnitudes, with F1 above 0.95 at 25 ns and 50 ns	Yücel et al. [25]	Adopt comparable timing features; adopt LOF as an additional baseline so results are comparable to published work
Unsupervised framing in prior work is a response to label scarcity in real deployments	Yücel et al. [25]	Use supervised multiclass classification, since simulation supplies ground-truth labels; this also yields a model with trainable weights, which plain LOF does not, and is therefore FedAvg-compatible without a non-standard aggregation scheme
Centralized, local-only and FedAvg comparison exposes the trade-off between federated performance and communication cost	Digregorio et al. [10]	Commit all three arms as the federated evaluation; retain FedProx as an extension

Finding	Source	Design decision
Random IID partitions and accuracy-only evaluation overstate federated performance	Shahsavari et al. [11]	Use structurally heterogeneous zone profiles, split by scenario and seed, and report worst-client performance rather than only the mean
Loss or manipulation of synchronisation causes clock divergence with a characteristic recovery profile	Kakade et al. [30]	Use divergence and post-attack recovery behaviour as attack-validation signals before any model is trained

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

The proposal defence panel returned six comments. Two concerns were each raised twice, which the team treated as indicating priority.

1. The proof of concept was assessed as crude, and the panel separately reiterated that the team needs to work on the PoC. This was the most emphasised item.
2. The evaluation matrix was assessed as requiring greater precision, and the panel separately noted that more effort should be made in the context of the evaluation metric.
3. The team was unable to explain the distinction between static and dynamic environments when questioned.
4. More in-depth background study was required.
5. The panel suggested better simulators, and add-on applications where the outcomes could be utilised.
6. The application perspective needed to be made stronger.

The panel's overall assessment recorded the project as a good niche project with a good direction, subject to the above.

3.2 Proposal Action Register

Table 3.1: Proposal action register

No.	Feedback / required change	Action taken	Status	Evidence
1	PoC is crude; students need to work on the PoC	Replaced the proposal-stage illustrative mock-up with an executable simulation: four-zone TSN topology with gPTP, three instrumented attack configurations, 15 seeded runs, and a feature extraction pipeline producing 4,740 labelled windows. The operator-facing surface is now explicitly labelled as a presentation mock-up and is no longer presented as the proof of concept.	Addressed	Ch. 6; Fig. 6.1; Sec. 6.9; Sec. 7.4
2	Evaluation matrix requires greater precision	Replaced the three-row metric summary with a full evaluation specification: per-class precision/recall/F1 and macro-F1, detection latency in sync intervals, false alarms per vehicle-hour, worst-client macro-F1, centralized-to-FedAvg gap, convergence rounds, update volume, and zone-attribution top-1 accuracy reported separately from detection. Accuracy removed as a headline measure, with class-imbalance justification.	Addressed	Sec. 1.9; Sec. 7.1; Sec. 9.5

No.	Feedback / required change	Action taken	Status	Evidence
3	Unable to explain static vs. dynamic environment	Both senses are now stated explicitly as load-bearing assumptions: the network environment is statically configured (A1), and training is static/one-shot rather than continuous (A2). The dependence of the detection approach on a statically known expected value is stated, and dynamic reconfiguration is recorded as an explicit limitation.	Addressed	Sec. 1.10 (A1, A2); Sec. 5.5; Sec. 8.11
4	More in-depth background study required	Literature review extended beyond the five sources presented at defence, recorded against methodology, results, limitations and resulting design decision; a research-to-decision table now links each finding to a concrete design choice; a comparison against the departmental FYP repository was added.	Addressed	Sec. 2.1–2.4; Sec. 2.8; Table 2.2
5	Better simulators; add-on applications where outcomes can be utilised	Simulation framework and version justified against alternatives in an architecture decision record. Consumption of detector output defined concretely: the vSOC incident queue, and the in-vehicle integrity decision consumed by fault-tolerant timing failover.	Partially addressed	Sec. 5.5; Sec. 1.8; Sec. 9.6

No.	Feedback / required change	Action taken	Status	Evidence
6	Application perspective needs to be stronger	Added deployment positioning (zone controller and central vehicle compute), the consuming stakeholders and their required evidence, and the deployment plan identifying what the detector's output drives.	Partially addressed	Sec. 1.3; Sec. 1.8; Sec. 9.6

3.3 Iteration Strategy

Development follows four time-boxed iterations, each ending in a demonstrable artefact rather than a document. The ordering is deliberately *environment* → *data* → *model* → *federation*, on the principle that no modelling decision can be validated before the data generation path is trustworthy. An explicit decision gate separates Iteration 02 from Iteration 03: the detection claim is tested centrally, on a single machine, before any federated infrastructure is built, so that a null feature result is discovered before it is built upon.

3.4 Iteration 1 Plan

Objective. Establish a reproducible simulation environment in which TSN timing attacks produce measurable, repeatable effects.

- **Selected features:** zonal topology construction; gPTP synchronisation; attack injection harness; seeded reproducible execution.
- **Deliverables:** `CarazoneZonalNetwork.ned`, `omnetpp.ini` with four named configurations, executable simulation.
- **Evidence:** topology rendering, benign vs. attack offset traces across multiple seeds.
- **Exit criterion:** an injected attack produces a timing deviation distinguishable from the benign baseline, reproducibly.
- **Status: complete.**

3.5 Iteration 2 Plan

Objective. Convert raw simulation output into a labelled, windowed, protocol-native feature dataset.

- **Selected features:** passive signal capture; windowing; feature computation; labelling with zone and class attribution; dataset export.
- **Deliverables:** `extract_features.py`, `features.csv`.
- **Evidence:** 225 vectors across 15 runs; 4,740 windows; class and per-zone distribution.
- **Exit criterion:** a labelled dataset regenerable from configuration and seed.
- **Status: complete.** Labelling is correctly attributed to the victim zone only; the target zone is rotated across runs so that every zone appears as both victim and bystander (Section 7.4).

3.6 Work Breakdown and Responsibilities

Table 3.2: Work breakdown by member

Area	Adeena Zaib	Khaula Bilqees
Setup	OMNeT++/INET zonal topology; model architecture	TSN-native signal instrumentation; federated learning environment setup
Data generation	Attack traffic simulation, all scenarios, per zone	Feature specification review; labelling scheme
Federated learning	Per-zone dataset packaging for clients	Federated client/server implementation
Training and evaluation	Attack-specific ground-truth verification	Model training; hyperparameter tuning; metrics collection
Shared	Literature review; architecture diagrams; report writing	Literature review; architecture diagrams; report writing

Each member is responsible for being able to explain, modify and defend the complete architecture, not only their primary component. No member's ownership is limited to documentation, presentation or manual testing.

3.7 Progress Against Plan

Table 3.3: Planned milestones versus actual progress

Milestone	Planned outcome	Status	Note
Iteration 01	Simulation environment; attacks instrumented	Complete	Four named configurations implemented
Iteration 02	Traffic generation; feature pipeline; labelled per-zone datasets	Complete	Cross-zone residual feature outstanding
Iteration 03	Federated learning pipeline; initial training runs	Not started	Preceded by residual feature and zone heterogeneity (Section 3.8)
Iteration 04	One-shot federated training; evaluation metrics collected	Not started	n/a

3.8 Delays, Deviations and Recovery Plan

D1. Detection model not yet implemented. The feature dataset is currently the end of the implemented pipeline. *Cause:* Iterations 01–02 absorbed more effort than planned, primarily in instrumenting protocol-internal signals that are not exposed by default. *Recovery:* a centralized detector is scheduled as the first Iteration 03 task, deliberately ahead of any federated infrastructure, so that the feature claim (SC-2) is tested before further work depends on it.

D2. Zone symmetry. All four zones of the Iteration 1 topology carry identical traffic and produce an identical number of feature windows. *Consequence:* federated, centralized and local-only training would converge to equivalent models, making the federated comparison (SC-4) null by construction. *Recovery:* the five-zone reference topology (Section 5.6) builds asymmetry into the structure itself, with between three and seven endpoint ECUs per zone and distinct traffic classes by zone function, giving quantity, feature and label skew by design. Differentiated per-zone traffic profiles and the measurement of the resulting skew remain Iteration 03 tasks, before any federated training run.

D3. Cross-zone residual feature added after mid-iteration review. The feature set initially computed statistics within each zone in isolation, with no comparison between a zone and its siblings. *Consequence:* the implemented features were timing statistics that a generic detector could also compute, which did not support the protocol-native claim

SC-2 is designed to test. *Resolution:* the extraction loop was restructured so that the observation window is the outer iteration, making every zone's controller offset available for the same window, and two residual features were added. The ablation in Section 7.5 confirms these carry the detection signal. *Closed.*

D4. Deviations from the approved proposal. Three deviations are recorded: the number of federated clients changed from three to four and then to five, in each case following the topology design; team composition is two members rather than the three assumed by the proposal's ownership model, with responsibilities reallocated accordingly (Section 3.6); and the attack surface was broadened to include FRER (IEEE 802.1CB) stream-redundancy manipulation alongside gPTP synchronisation attacks, which the approved proposal did not name. FRER remains committed scope. It is not represented in the Iteration 1 tree dataset because that topology has no redundant paths, but it has since been simulated on the five-zone ring, where replication and elimination were confirmed and two stream-redundancy attacks were run (Section 6.10). Feature extraction for FRER is scheduled in Section 9.3.

Chapter 4

Software Requirements

4.1 User Interactions / Use Cases

The system has four actors:

- **Validation Engineer (human).** Configures scenarios, executes runs, generates datasets, trains and evaluates models, inspects results.
- **vSOC Analyst (human).** Reviews attributed alerts, triages incidents.
- **Zone Controller (system).** Observes local TSN signals, extracts features, performs inference, emits alerts, participates in federated rounds.
- **Central Vehicle Compute (system).** Aggregates model parameters, distributes the global model, receives and logs alerts.

Primary use cases: UC-1 Configure and execute scenario; UC-2 Inject attack with ground truth; UC-3 Extract feature dataset; UC-4 Train local model; UC-5 Aggregate federated model; UC-6 Classify observation window; UC-7 Emit attributed alert; UC-8 Review incident queue; UC-9 Evaluate and export metrics.

4.2 Use Case Diagram

4.3 Detailed Use Cases

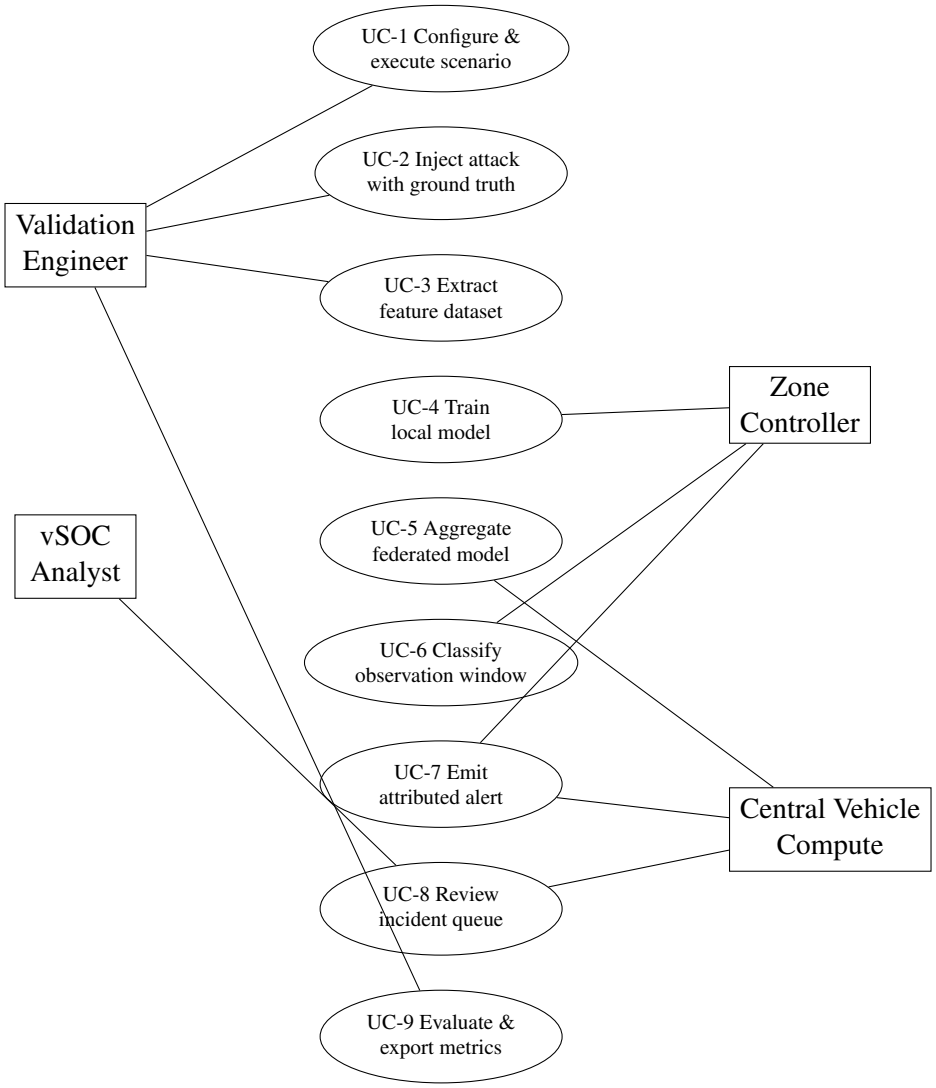


Figure 4.1: Use case diagram. Human actors are shown left, system actors right.

UC-2: Inject attack with ground truth	
Actor	Validation Engineer
Precondition	A valid topology and benign baseline configuration exist
Trigger	Engineer selects an attack configuration and seed
Main flow	(1) Engineer selects attack class, target zone, start time, intensity and seed. (2) Simulation executes with injection active over the specified interval. (3) Harness records, for every observation window, the attack class and the set of affected zones. (4) Raw per-node vectors and the ground-truth record are written to the results directory.
Postcondition	A seeded, reproducible run with labels attributable to class, window and zone
Exception	If the attack produces no measurable deviation from baseline, the run is flagged for parameter review rather than silently retained

UC-6: Classify observation window	
Actor	Zone Controller
Precondition	A trained global model is deployed locally
Trigger	A complete observation window of local TSN signals is available
Main flow	(1) Controller computes the feature vector for the window. (2) Vector is passed to the local copy of the global model. (3) Model returns a class and a confidence value. (4) If the class is not benign for k consecutive windows, an alert is raised.
Postcondition	Either no action, or an attributed alert
Exception	If no valid model is available, the rule-based fallback detector operates and the degraded state is made visible

4.4 Functional Requirements

4.4.1 Module 1: Simulation and Attack Injection

- **FR-01.** The system shall instantiate a configurable multi-zone TSN topology comprising a grandmaster, a backbone switch, N zone controllers, endpoint ECUs and a central vehicle compute node.
- **FR-02.** The system shall synchronise all time-aware nodes using gPTP.

- **FR-03.** The system shall execute reproducibly: a given configuration and seed shall regenerate the same scenario structure and ground-truth labels.
- **FR-04.** The system shall generate benign traffic and at least two attack classes against TSN synchronisation.
- **FR-05.** The system shall record, for every injected attack, the attack class, the affected zone(s), and the start and end time of the attack interval.

4.4.2 Module 2: Feature Extraction

- **FR-06.** The system shall convert raw per-node simulation output into fixed-size observation windows.
- **FR-07.** The system shall compute, per window per zone, protocol-native timing features including clock offset statistics, sibling offset residual, and synchronisation interval residual.
- **FR-08.** The system shall label each window with its attack class and shall attribute the label to the affected zone(s) only.
- **FR-09.** The system shall export the labelled dataset in a documented schema with a stable column contract.
- **FR-10.** Feature extraction shall be passive and shall not alter forwarding behaviour in the simulated network.

4.4.3 Module 3: Detection and Federation

- **FR-11.** The system shall train a multiclass detection model on the labelled feature dataset.
- **FR-12.** The system shall support training in three configurations, namely local-only per zone, centralized pooled, and federated (FedAvg), using an identical model architecture and identical scenario splits.
- **FR-13.** The federated coordinator shall receive model parameters only; raw feature records shall not leave the client.
- **FR-14.** The system shall classify an observation window into benign or a specific attack class and shall return a confidence value.
- **FR-15.** The system shall provide a rule-based fallback detector that operates when no valid model or coordinator is available, and shall make the degraded state visible.

4.4.4 Module 4: Alerting and Evaluation

- **FR-16.** The system shall emit alerts containing attack class, affected zone, time window, confidence and supporting feature values.
- **FR-17.** The system shall present alerts through an operator-facing incident queue supporting filtering by attack class, severity and status.
- **FR-18.** The system shall export machine-readable evaluation metrics linked to the configuration, seed and model version that produced them.

4.5 Non-Functional Requirements

4.5.1 Reliability

- **NFR-01.** A documented configuration and seed shall regenerate the same scenario structure and ground-truth labels on a clean environment.
- **NFR-02.** Loss of the coordinator or of a valid model shall produce a visible fallback state rather than silent cessation of monitoring.

4.5.2 Usability

- **NFR-03.** An operator shall be able to execute a scenario, extract features and inspect results from documented commands without modifying source code.
- **NFR-04.** Each alert shall carry sufficient attribution for an analyst to identify the affected zone and time window without consulting raw simulation output.

4.5.3 Performance

- **NFR-05.** Local classification of one observation window shall complete within one observation interval on the target workstation.
- **NFR-06.** Detection shall occur within a bounded number of synchronisation intervals from attack onset, and this latency shall be reported in sync intervals rather than wall-clock seconds.

4.5.4 Security

- **NFR-07.** Raw feature records shall not be transmitted to the coordinator; only model parameters and training metadata shall be exchanged.
- **NFR-08.** Passive monitoring shall not increase protected-stream 99th-percentile latency by more than 5% in matched enabled/disabled simulation runs.

4.6 Acceptance Criteria

Req.	Objective acceptance check
FR-03	Two executions with identical configuration and seed produce byte-identical ground-truth label files.
FR-05	For each attack run, the recorded attack interval matches the interval over which the timing deviation is observable in the raw trace.
FR-07	For a benign run, the sibling offset residual remains within the documented benign envelope for all zones across all seeds.
FR-08	For an attack run targeting zone z, windows in zone z carry the attack label and windows in all other zones carry the benign label.
FR-10	Protected-stream latency distributions from matched monitoring-enabled and monitoring-disabled runs are statistically indistinguishable within the NFR-08 bound.
FR-12	All three training configurations run to completion on identical splits and produce comparable metric files.
FR-13	Inspection of coordinator-bound messages confirms that no feature record fields are present.
NFR-06	Measured detection latency is lower than the latency at which accumulated offset exceeds the configured schedule margin, across all seeds.

4.7 Requirements Traceability

Table 4.4: Requirements traceability matrix

Req.	Module	Design element	Implementation	Test
FR-01	M1	Sec. 5.1, Fig. 6.1	`CarazoneZonalNetwork.ned`	TC-01
FR-02	M1	Sec. 5.3	gPTP configuration in `omnetpp.ini`	TC-02
FR-03	M1	Sec. 5.5	Seeded run configurations	TC-03
FR-04	M2	Sec. 5.10	`DelayAttack`, `DriftAttack`, `SyncStarvation`	TC-04
FR-05	M2	Sec. 5.9	Ground-truth record	TC-05
FR-06	M3	Sec. 5.10, Fig. 5.6	`extract_features.py`	TC-06
FR-07	M3	Sec. 5.8	`extract_features.py`	TC-07
FR-08	M3	Sec. 5.9	`extract_features.py`	TC-08 (DEF-01)
FR-09	M3	Table 5.6	`features.csv`	TC-09
FR-11–15	M4, M5	Sec. 5.1–5.4	Planned, Iteration 03	Planned
FR-16–18	M6	Sec. 5.1	Planned, Iteration 04	Planned

Chapter 5

System Design

5.1 System Architecture

The system is organised into five layers with explicit interface contracts between them. Layer separation is deliberate: the simulation layer is replaceable by captured traces from a hardware testbed, and the detection layer is replaceable by a different model, without either change propagating beyond its own interface.

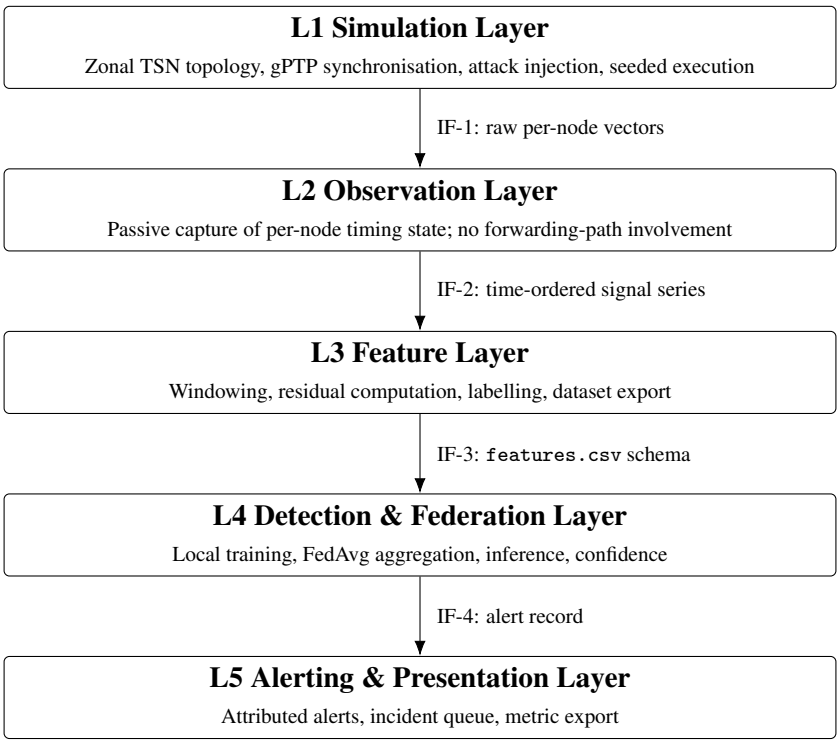


Figure 5.1: Layered system architecture with interface contracts.

5.2 Detection Methodology and Deployment View

Figure 5.1 describes the system as a stack of layers. Figure 5.2 describes the same design from the deployment side: where each stage of the detection pipeline runs inside the vehicle, and what crosses the boundary between the zonal controllers and the central vehicle compute unit. It also separates the two phases of operation, training and real-time detection, because they use the same components but move different data.

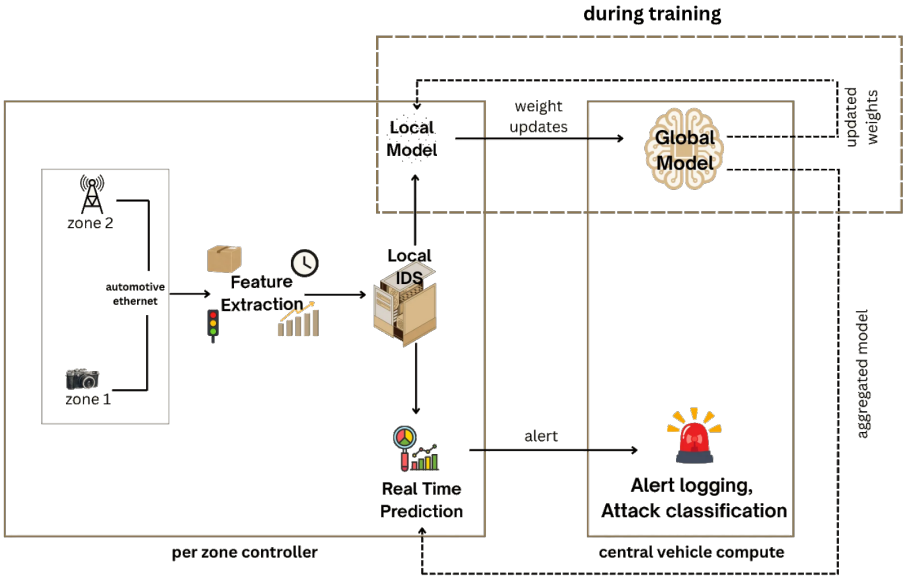


Figure 5.2: Detection methodology and deployment view. Solid arrows show the real-time detection path inside each zonal controller and the alert sent to the central vehicle compute unit. Dashed arrows, enclosed in the "during training" region, show the federated training exchange: local weight updates flow to the global model, and the aggregated model returns to each zone controller. The diagram shows two zones for clarity; the reference topology of Section 5.6 has five.

Real-time detection path

Traffic from the endpoint devices in a zone reaches the zonal controller over automotive Ethernet. The feature extraction stage segments this traffic and the associated gPTP timing state into fixed-size observation windows and computes the per-window statistics and residuals defined in Section 5.9. The resulting feature vector is passed to the local IDS, which classifies the window using the deployed model and produces a real-time prediction. When the prediction indicates an attack, an alert is sent to the central vehicle compute unit, where it is logged and the attack class is recorded. This placement follows decision AD-6: the zone controller is the lowest point in the hierarchy that observes

enough ports at once to compute the sibling-offset residual, so detection runs there rather than at the endpoint ECUs.

Federated training path

During training, each zonal controller uses the feature records it has produced to train a local model. Only the resulting model weights are sent to the central vehicle compute unit, where they are combined into a global model by FedAvg. The updated weights are returned to every zone controller, and the aggregated model is the one deployed in the local IDS for real-time prediction. Feature records never leave the zone, which is the constraint stated in NFR-07 and the boundary defined by interface IF-5. Federation across zones, with the vehicle's own central compute unit as aggregator, is decision AD-3; training is static and one-shot (AD-4), so the dashed path in Figure 5.2 is inactive once the model has been deployed.

Relation to the layered architecture

The two views describe the same system. The zone-controller stages of Figure 5.2 (feature extraction, local IDS, local model) correspond to layers L3 and L4 of Figure 5.1, the weight and model exchange is the L4 federation path over IF-5, and the alert flowing to the central unit is the IF-4 alert record consumed by L5. The simulation and observation layers (L1, L2) sit upstream of the diagram: in this project they stand in for the automotive Ethernet traffic and timing state that a real zone controller would observe.

5.3 Architectural Components

Component	Layer	Responsibility
Topology model	L1	Declares nodes, links and per-node clock configuration
Scenario manager	L1	Schedules attack events at configured times and targets
gPTP instances	L1	Distribute time from the grandmaster through the relay tree
Signal recorder	L2	Records per-node clock state at sample points
Window builder	L3	Segments series into fixed-size observation windows

Component	Layer	Responsibility
Feature computer	L3	Computes per-window statistics and residuals
Labeller	L3	Assigns class and zone attribution from the ground-truth record
Local trainer	L4	Trains the shared model architecture on a client's local dataset
Aggregator	L4	Performs FedAvg over received parameters
Inference engine	L4	Classifies windows using the deployed global model
Fallback detector	L4	Rule-based detection when no valid model is available
Alert builder	L5	Constructs attributed alert records
Incident queue	L5	Operator-facing triage surface

5.4 Interfaces and Dependencies

ID	Interface	Contract
IF-1	Simulation → Observation	Per-node recorded vectors in the simulator's result format; one vector per node per run; node identity encoded in the vector name
IF-2	Observation → Feature	Time-ordered (t, value) series per node, with run identifier and seed
IF-3	Feature → Detection	`features.csv` column contract (Table 5.6); one row per window per zone; stable column order; documented units
IF-4	Detection → Alerting	Alert record: {class, confidence, zone, window start, window end, run id, supporting feature values}
IF-5	Client ↔ Coordinator	Model parameters and training metadata (sample count, round index). Feature records are explicitly excluded from this interface (NFR-07)

External dependencies. OMNeT++ 6.4.0 (simulation kernel, Qtenv/Cmdenv), INET 4.7.0 [31] (TSN and gPTP modules), Python 3 with scientific stack for extraction and

training, and a federated learning framework for the aggregation layer. Versions are pinned in Section 8.8.

5.5 Architecture Decisions and Rationale

Table 5.3: Architecture decision record

ID	Decision	Alternatives considered	Rationale
AD-1	Use OMNeT++/INET as the simulation framework	Build a custom simulator; use a hardware TSN testbed; use a generic network simulator without TSN modules	INET provides gPTP, time-aware shaping and clock-drift modelling as maintained components, and its TSN capabilities have been verified against TSN hardware measurements in published work. Building a simulator would consume the project budget without advancing the research question. A hardware testbed is unavailable and its procurement risk is unacceptable within the timeline.
AD-2	Supervised multiclass classification	Unsupervised (LOF, autoencoder, Deep SVDD); two-stage hybrid	Simulation supplies ground-truth labels, removing the label-scarcity constraint that motivates unsupervised framing in the comparable literature. A parametric model has trainable weights and is therefore FedAvg-compatible without a non-standard aggregation scheme; LOF is not. LOF is retained as a comparison baseline for comparability with published results.

ID	Decision	Alternatives considered	Rationale
AD-3	Federate across zones within one vehicle	Federate across vehicles/fleet	The aggregation server is the vehicle's own central compute unit. Within-vehicle federation is motivated by backbone bandwidth, not inter-organisational privacy: periodic fixed-size parameter exchange is a bounded, schedulable traffic pattern, whereas continuous trace upload is unbounded best-effort traffic competing with protected streams.
AD-4	Static, one-shot training	Continuous/online federated learning	Online retraining in deployment requires live ground-truth labelling, which has no solution within scope. Assumption A2.
AD-5	Static network environment assumption	Support runtime reconfiguration	The detection approach computes residuals against an expected value known from configuration. This is available *a priori* only when the configuration is static. Supporting dynamic reconfiguration requires distinguishing legitimate reconfiguration from attack, which is a separate research problem. Assumption A1.

ID	Decision	Alternatives considered	Rationale
AD-6	Detector positioned at the zone controller	Position at endpoint ECU; position centrally	The primary features require simultaneous visibility of multiple ports: the sibling offset residual cannot be computed by a node that observes only its own clock. The zone controller is the lowest point in the hierarchy with the required visibility.
AD-7	Flat CSV dataset with a documented column contract rather than a relational schema	Relational database with entity-relationship model	The data is a single homogeneous observation table with no multi-entity relational structure. A relational model would add no referential integrity benefit and would not reflect the domain. Section 5.9 therefore specifies a column contract rather than an ERD.

5.6 Reference Topology Design

The Iteration 1 results reported in Chapter 7 were produced on a four-zone tree topology, described in Section 6.6 and retained as `topology_basic`. That topology was built to be minimal: it was enough to instrument gPTP, inject delay attacks and test the cross-zone residual claim, and deliberately no more. It has two properties that bound what can be asked of it. Each zone reaches the backbone over a single path, so FRER cannot be configured, and all four zones carry identical traffic, which is recorded as defect DEF-02.

A second topology, `topology5`, has since been designed and generated to remove both limits. It is the reference topology for the remainder of the project. It has been validated structurally and has produced the attack simulation results of Section 6.10, but not yet a labelled feature dataset, so the detection results of Chapter 7 remain those of the tree.

5.6.1 Basis of the design

No vehicle manufacturer publishes harness or ECU-placement data, so the topology is built as a representative reference rather than a copy of a production vehicle. Three published sources inform it. The zone-based in-vehicle architecture evaluated in *Sensors* [12] supplies the organising principle: ECUs are grouped into zones by physical location, attached to the nearest zone controller, with a central compute unit as the inter-zone hub. The TSN anomaly-detection assessment framework of Meyer, Häckel, Lübeck, Korf and Schmidt [13] supplies a precedent for a ring-based zonal topology carrying corner lidar, front and rear cameras, infotainment, a connectivity gateway, a gPTP master and an ADAS unit, modelled in OMNeT++/INET. Industry material on zonal E/E architecture, including the IEEE 802 zonal presentation and Excelfore's TSN wheel-sensor work, supplies the four-corner-plus-cabin arrangement and the use of TSN for time-critical traffic with network redundancy. The device list is the project's own representative model and is not claimed to match any production vehicle.

5.6.2 Zone partitioning

The vehicle is divided into five zones: front-left (FL), front-right (FR), rear-left (RL), rear-right (RR) and a centre/cabin zone (C). Each zone has one TSN switch and one zonal controller host, and the controller is the node that runs the local detector and acts as the federated client. Devices are assigned to zones by three rules applied in order: physical proximity, so a device connects to the switch of the zone in which it is mounted; safety separation, so safety-critical actuators such as steering and brake are kept apart from infotainment and external-facing nodes; and a load limit, so a zone's uplink load stays within the capacity of its links, with devices moved or link speed revisited if it does not.

The model contains 25 endpoint ECUs, five zonal controllers, a central compute unit, a connectivity gateway and a gPTP master clock.

Table 5.4: Zone composition of the reference topology

Zone	ECUs	Examples
Front-left	7	lidar, radar, front camera, left mirror camera, steering, brake, front ultrasonic
Front-right	6	lidar, radar, right mirror camera, powertrain, ultrasonic, headlamp
Rear-left	3	lidar, radar, ultrasonic
Rear-right	4	lidar, radar, rear camera, V2X unit

Zone	ECUs	Examples
Centre/cabin	5	infotainment, instrument cluster, driver-monitoring camera, audio amplifier, GNSS receiver

The distribution is deliberately uneven. The front zones carry dense sensing and control traffic, the rear zones are lighter, and the cabin zone carries mainly multimedia and display traffic. The ECU count was chosen for this project rather than copied from the source architecture: enough devices per zone to give each federated client a meaningful local dataset, while keeping the simulation maintainable.

5.6.3 Interconnect

Within a zone, devices connect to the zone switch in a star. Between zones, the four corner switches form a ring (FL, FR, RR, RL) and the centre switch is dual-homed to both front switches. Every zone therefore has two link-disjoint routes to the centre, which is precisely the property that Frame Replication and Elimination for Reliability requires and which the earlier tree topology could not provide. Ring links and ADAS sensors run at 1 Gbps; lower-rate ECUs run at 100 Mbps. Link lengths of 0.5 to 3.5 m are assumed values chosen to be plausible for a passenger vehicle and are kept below the 15 m reach of automotive Ethernet over unshielded twisted pair.

The ring is the primary topology because it enables FRER-based detection features and path-change behaviour in gPTP. A star topology over the same device inventory is retained as a baseline for comparison, since a star offers only one path per zone and therefore isolates what the redundancy actually contributes.

5.6.4 Time synchronisation

A gPTP grandmaster is attached to the centre switch. Synchronisation follows a static tree: grandmaster, centre switch, front switches, rear switches, then end devices. The two ring links not on this tree (FL to FR, and RL to RR) carry no synchronisation messages. The simulator's gPTP implementation assigns master and slave ports statically and has no best-master-clock algorithm, so a backup grandmaster is not modelled within the current scope. gPTP and FRER are the two TSN signals used for detection.

5.6.5 Generation and reproducibility

The network description (`.ned`), the simulation configuration (`omnetpp.ini`) and the validation checks are all generated from a single Python table of devices, link speeds and link lengths (`gen_topology.py`). Editing the table and re-running the generator regenerates both simulator files, so the model cannot drift out of step with its own specification, and the generation step runs in the project's continuous integration pipeline (Section 8.5).

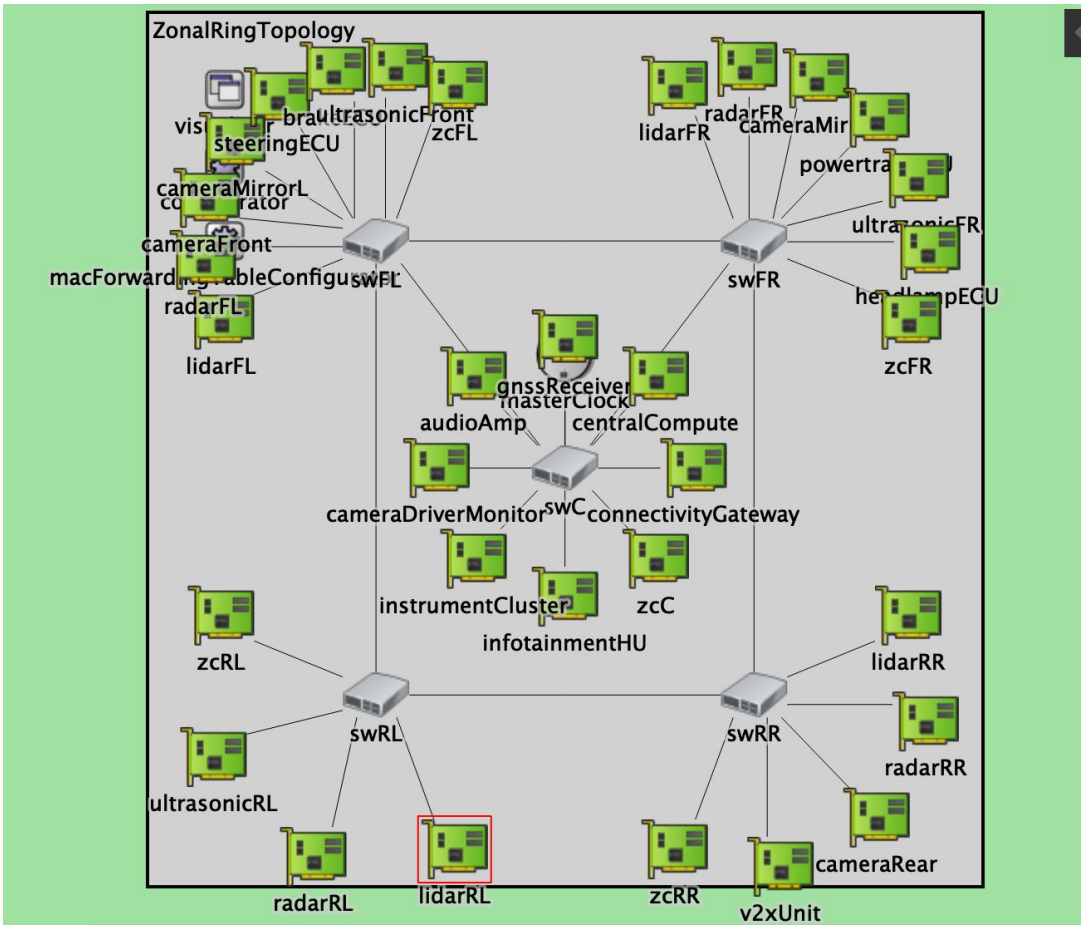


Figure 5.3: The five-zone ring reference topology (`ZonalRingTopology`) as rendered in the simulator. The four corner switches `swFL`, `swFR`, `swRR` and `swRL` form the ring; the centre switch `swC` is dual-homed to both front switches. Every zone therefore has two link-disjoint routes to the centre, which is the property FRER requires. Each zone switch carries its own zonal controller (`zcFL`, `zcFR`, `zcRL`, `zcRR`, `zcC`), which is the node that acts as a federated client, together with the endpoint ECUs of that zone. The grandmaster (`masterClock`) and the central compute unit are attached to `swC`.

5.6.6 Validation

The topology is checked at three levels.

Structural, automated. The generator verifies that the graph is connected, that each endpoint has exactly one switch port, that cutting any single ring or uplink link partitions nothing, and that every link is shorter than 15 m. The current model passes all of these checks. The partition check is the one that matters for FRER: it is the machine-checkable form of the disjoint-path requirement.

Physical plausibility, manual. Each device is checked against the placement rules: sensors connect to the switch of the zone in which they are mounted, cabin devices connect to the front or centre zone, and safety-critical ECUs are kept separate from infotainment. Each placement is justified by a source or a stated reason.

Network behaviour, simulation. The third level is specified here as the acceptance criteria for the benign baseline run, which was the first task of Iteration 03c (Section 9.3). The topology is accepted as behaviourally sound when all time-triggered flows meet their deadlines, link utilisation stays below approximately 70%, gPTP converges within the target bound, and FRER flows survive a cut ring link. Stating these criteria before the run is deliberate: it fixes what counts as a successful topology in advance, so that the baseline run tests the design rather than being read as a description of whatever it happens to produce. The baseline has since been run (Section 6.10): there was no packet loss, gPTP converged, and FRER-protected delivery survived a crashed ring switch. The deadline and link-utilisation criteria have not yet been recorded.

5.6.7 Expected heterogeneity

Each zonal controller acts as a federated client whose data is the traffic passing through its zone switch. The topology is designed so that three kinds of non-IID behaviour can arise across those clients: quantity skew, since zones differ in the number of flows and therefore in the number of samples; feature skew, since traffic classes, packet sizes and rates differ by zone and gPTP delay grows with hop distance from the grandmaster; and label skew, since attack exposure differs by zone, with V2X-related attacks reaching the rear-right zone and gPTP attacks affecting deeper zones more strongly.

This is the designed remedy for defect DEF-02, which records that the four zones of the earlier topology carry identical traffic and would make the federated comparison null by construction. The remedy is a design property until it is measured. It will be quantified by comparing per-zone feature distributions, using the Kolmogorov–Smirnov statistic or the Wasserstein distance, and per-zone label distributions, using the Jensen–Shannon divergence. Until those numbers exist, DEF-02 is reduced from open to partially addressed rather than closed.

5.6.8 Limitations

Four limitations are recorded against this topology. The device list and link lengths are representative rather than taken from a manufacturer's vehicle. The model carries five benign application flows, one representative per zone, but the heterogeneity described above is still a design property and not a measured one. gPTP is statically configured with no backup grandmaster, so grandmaster-failover behaviour cannot be exercised. Some zone switches have ten or eleven ports, which is higher than a typical automotive TSN switch; they are best read as zonal controllers with integrated switching rather than as discrete switches.

5.7 Design Evolution

Six design changes have occurred since the approved proposal, each with its justification:

Element	**Proposal → Current**	**Justification**
Number of federated clients	Three → Four	Followed from the zonal topology design, which maps a client to a physical zone rather than to an arbitrary partition. Client count remains a configuration parameter.
Attack classes	Sync manipulation and delay injection → delay injection, sync starvation, drift (runtime form not realisable)	Sync starvation is a concrete realisation of synchronisation manipulation. Drift manipulation could not be realised as a runtime attack, and applied from $t = 0$ it produced no zone-localisable offset signature (Section 7.6). On the ring, a constant drift is visible in the servo's rate ratio (Section 6.10), so drift is retained as a documented negative result for the offset residual and as a candidate for rate-ratio features.

Element	Proposal → Current	Justification
Reference topology	Four-zone tree → five-zone ring with dual-homed centre	The tree was minimal by design and sufficient to test the residual claim, but it offers one path per zone and so cannot carry FRER. The ring gives every zone two link-disjoint routes to the centre. Specified in Section 5.6; Iteration 1 results in Chapter 7 remain those of the tree.
Federated client count	Four → five	Follows the reference topology, which has five zones rather than four. The client count is a property of the topology, not an independent parameter.
FRER-based detection	Not in proposal → committed scope, simulated on the ring	Added to the attack surface after the proposal, to test whether the cross-zone residual principle generalises beyond the synchronisation plane. The single dependency, the absence of disjoint paths in the Iteration 1 tree, is resolved by the five-zone ring topology, on which 802.1CB has been configured and two stream-redundancy attacks simulated (Section 6.10). The FRER feature family is scheduled in Section 9.3.
Team composition	Three-member ownership model → two members	Responsibilities reallocated across two members (Section 3.6); each member retains a distinct primary implementation area.

5.8 Domain Model

The domain is described by seven concepts. The synchronisation tree structure is central: the sibling relationship between zones at the same depth is what makes the residual feature computable, and its absence is what makes a grandmaster-level perturbation undetectable by this approach.

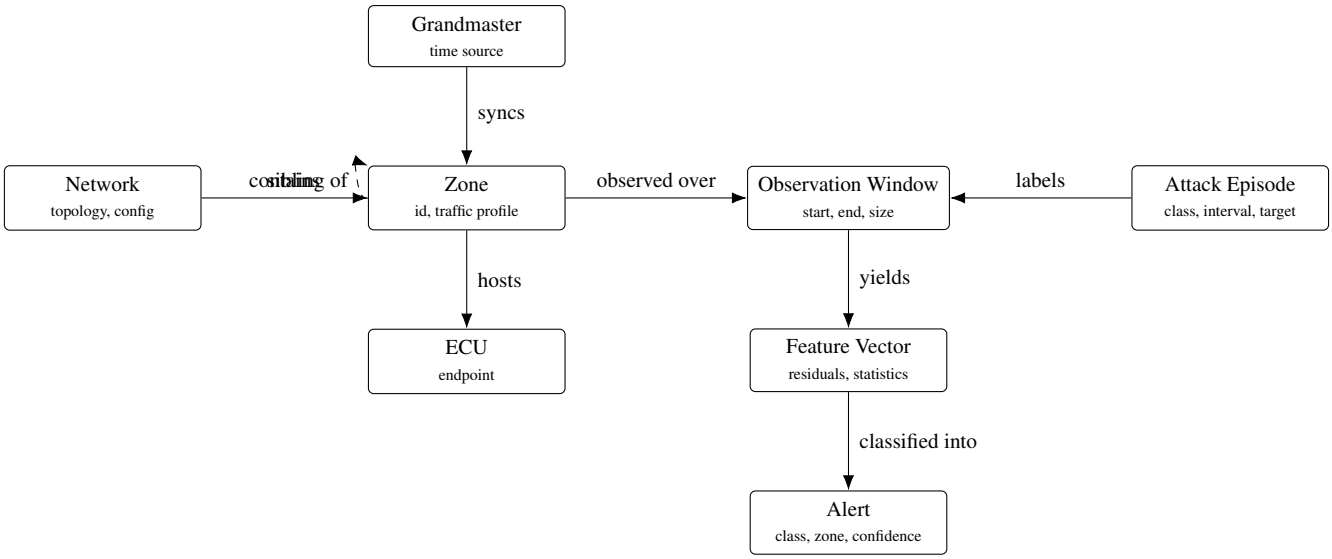


Figure 5.4: Domain model. The dashed self-relation on Zone is the sibling relationship within the synchronisation tree, on which the residual feature depends.

5.9 Data Design

The system does not use a relational database. Its persistent data is a single homogeneous observation table together with run metadata, so the data design is specified as a column contract rather than an entity-relationship model (decision AD-7).

Table 5.6: `features.csv` column contract

Column	Type	Unit	Description
Identity and labelling			
`run`	string	n/a	Run identifier assigned by the simulator, unique per configuration and seed
`config`	string	n/a	Configuration name: `Benign`, `DelayAttack`, `DriftAttack` or `SyncStarvation`
`zone`	int	n/a	Zone index, $0 \ldots N-1$
`target_zone`	int	n/a	Zone targeted by the attack in this run; empty for benign runs. Ground truth for attribution, and excluded from every feature set because it is equivalent to the label

Table 5.6 continued from previous page

Column	Type	Unit	Description
`window_start`	float	s	Simulation time at window start; window length and stride are extraction parameters
`label`	int	n/a	Class id: 0 benign, 1 delay, 2 drift, 3 starvation
Zone-controller features, prefix `ctrl_`			
`n_events`	float	count	Number of clock-change events in the window; zero is retained rather than dropped, as absence of events is itself the signature of sync starvation
`mean_offset`	float	s	Mean clock offset against ideal time
`std_offset`	float	s	Standard deviation of offset
`max_abs_offset`	float	s	Maximum absolute offset
`offset_range`	float	s	Peak-to-peak offset
`mean_abs_step`	float	s	Mean absolute correction step between consecutive events
`max_abs_step`	float	s	Largest single correction step
`drift_slope`	float	s/s	Linear slope of offset over the window
Zone aggregates over controller and member ECUs, prefixes `zone_mean_` *and* `zone_std_`			
`zone_mean_{f}`	float	n/a	Mean of feature f across all nodes in the zone, for each of the eight features above
`zone_std_{f}`	float	n/a	Standard deviation of feature f across all nodes in the zone
Planned, Iteration 03			
`sibling_residual`	float	s	Controller mean offset minus the median controller mean offset of all other zones in the same window. **Not yet implemented**
`sibling_n_{f}`	float	count	Number of sibling zones that contributed to the residual for feature f in this window. Recorded so that a residual computed against a reduced set of siblings can be identified rather than silently trusted

Offset definition. Node offset is derived from the simulator's recorded clock trace as $\text{offset}(t) = \text{clock_time}(t) - \text{simulation_time}(t)$, the node's error against the ideal timeline. Under correct gPTP operation this remains within a narrow band, punctuated by corrections at each synchronisation interval; each attack class perturbs that pattern differently.

Integrity rules. (R1) Every window belongs to exactly one run and one zone. (R2) A window carries an attack class label only if its zone is the designated victim of the run *and* the attack is active over the window. (R3) Train/test splits are formed at run and seed granularity, never at window granularity, because the extraction stride is shorter than the window length and adjacent windows therefore overlap.

5.10 Process / Behavior Design

Data flow

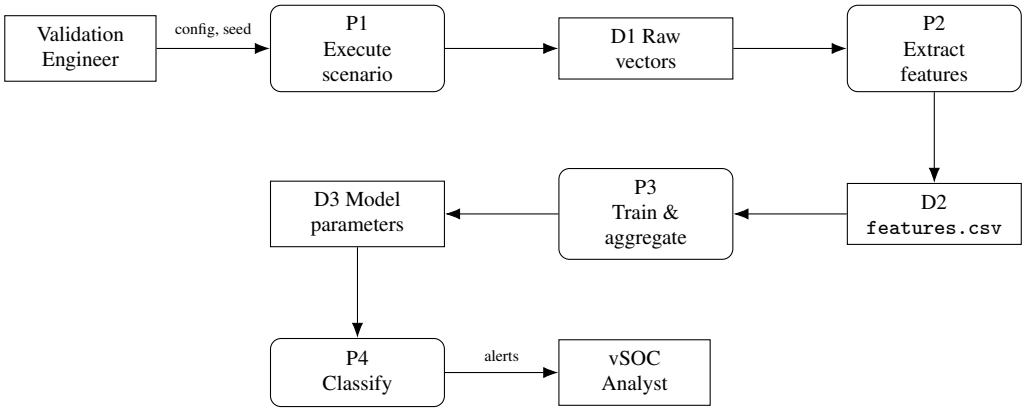


Figure 5.5: Level-1 data flow diagram.

Feature extraction activity

Detector state behaviour

Attack mechanism

The asymmetric delay attack exploits the symmetry assumption in the peer-delay computation. Under normal operation a node computes link delay as $D = ((t_4 - t_1) - (t_3 - t_2))/2$, which is correct only when the forward and reverse path delays are equal. An attacker who delays messages in one direction only introduces an asymmetry δ, producing a link-delay

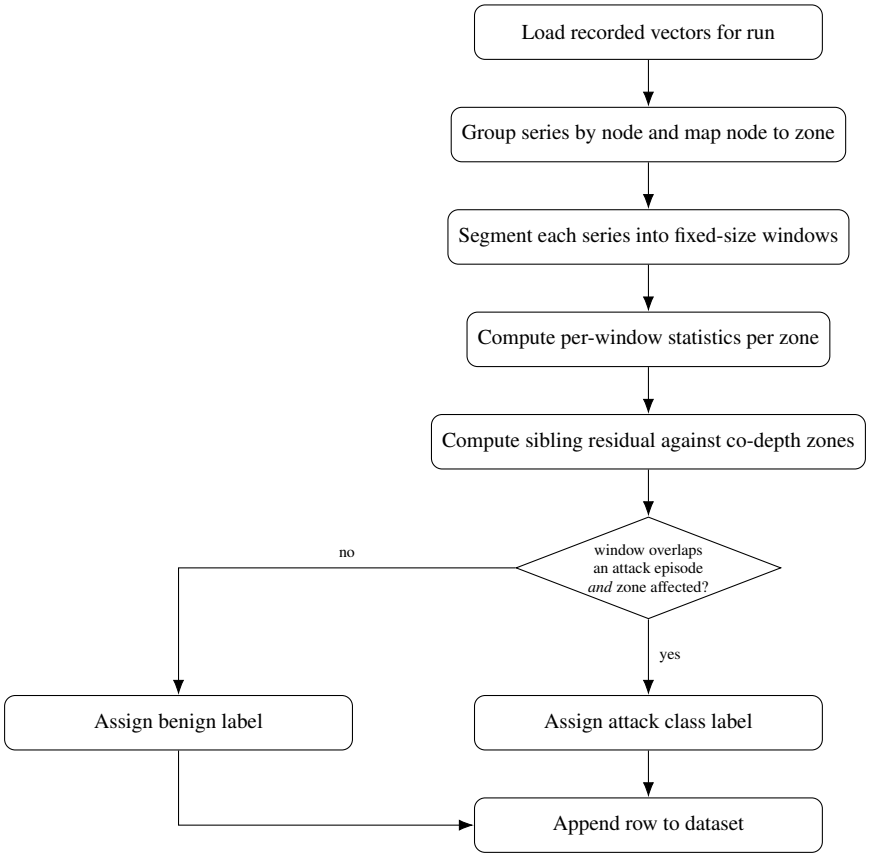


Figure 5.6: Feature extraction activity. The decision node implements integrity rule R2: a window is labelled with the attack class only when its zone is the run's designated victim and the attack is active. The sibling-residual step is planned for Iteration 03.

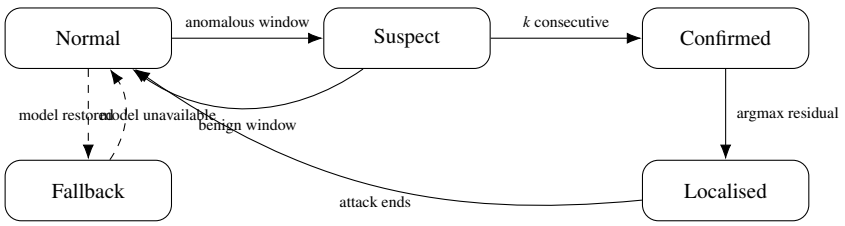


Figure 5.7: Detector state machine. The k-consecutive-window transition implements the detection-latency requirement NFR-06; the dashed transitions implement FR-15.

error of $\delta/2$ that propagates directly into the offset correction. The messages themselves remain authentic and unmodified, which is why message authentication does not mitigate this attack class.

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

Establish a reproducible zonal TSN simulation in which injected timing attacks produce measurable, repeatable effects, and convert the resulting raw output into a labelled protocol-native feature dataset. (Objectives O1–O3.)

6.2 Selected Use Cases / Features

UC-1 Configure and execute scenario; UC-2 Inject attack with ground truth; UC-3 Extract feature dataset. Requirements FR-01 through FR-10.

6.3 Design

Iteration 1 realises layers L1 to L3 of the architecture in Figure 5.1, together with the interface contracts IF-1 to IF-3 that join them. L1 is the simulation layer, which produces raw clock-offset vectors. L2 is the extraction layer, which converts those vectors into fixed windows and emits a feature row per window per zone. L3 is the dataset layer, which holds the labelled feature table. The detection and federation layers, L4 and L5, are deliberately out of scope for this iteration.

The design decision that shapes the iteration is the boundary at IF-2, between extraction and dataset. Everything above that boundary consumes a column contract (Table 5.6) rather than simulator output, so the detector never depends on OMNeT++ data structures. This is what makes the pipeline replaceable by a captured trace from real hardware in FYP-2 without changing the detection code, and it is the reason extraction was completed

and validated before any model was trained.

6.4 Implementation

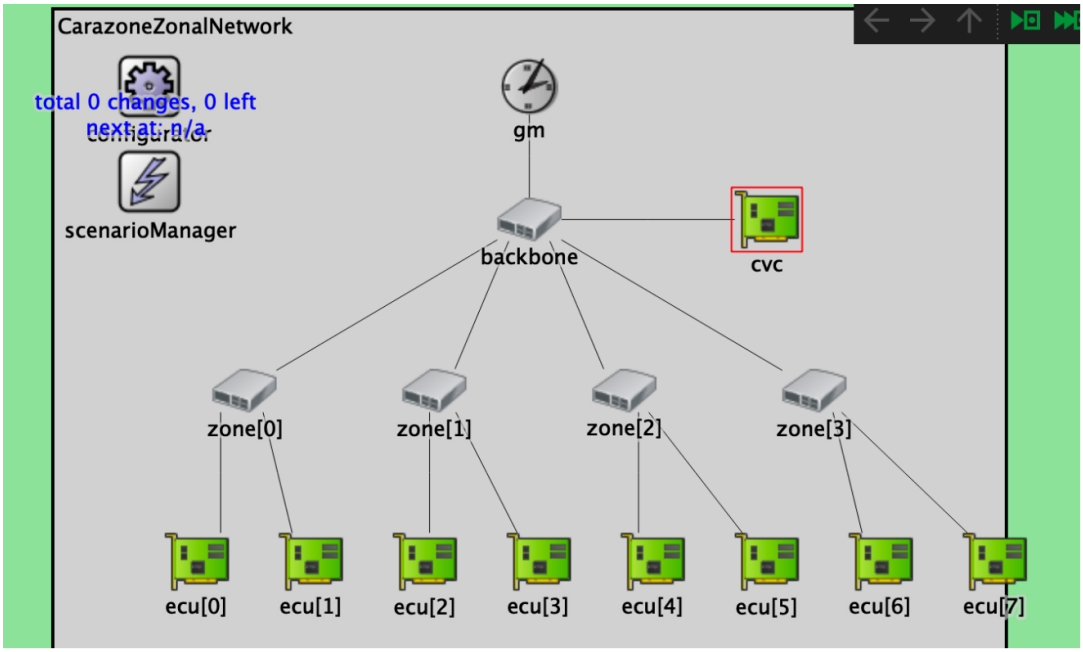


Figure 6.1: Implemented zonal topology: grandmaster, backbone switch, four zone controllers, eight endpoint ECUs and the central vehicle compute node.

The implemented topology comprises a grandmaster clock source, a backbone switch, four zone controllers, eight endpoint ECUs (two per zone), and a central vehicle compute node attached to the backbone. A configurator and a scenario manager handle address assignment and timed attack events respectively. All time-aware nodes participate in a single gPTP domain rooted at the grandmaster, giving a synchronisation tree of depth three in which the four zone controllers are siblings.

6.5 Completed Features / Module

- M1 Zonal Topology Simulation: complete
- M2 Attack Injection and Dataset: complete for three attack configurations
- M3 Feature Extraction Layer: complete, with DEF-01 open

6.6 Technical Implementation Details

Environment. OMNeT++ 6.4.0 with INET 4.7.0 [31], on macOS (Apple Silicon). The linked INET version is 4.7.0, which is the version whose `driftRate` parameter is not declared `@mutable`. That property is the reason the drift attack cannot be activated part-way through a run, and therefore the reason the drift result in Section 7.6 is reported as a limitation of the framework rather than a finding about drift detectability.

Key files.

- `CarazoneZonalNetwork.ned`: topology declaration: node types, links, and the submodule structure of the zonal network.
- `omnetpp.ini`: scenario configurations. Four named configurations are defined: `Benign`, `DelayAttack`, `DriftAttack` and `SyncStarvation`, together with clock drift parameters, gPTP settings and seed control.
- `carazone`: the built simulation executable.
- `extract_features.py`: the feature extraction pipeline.

Execution. Scenarios are executed headlessly, selecting a configuration by name:

```
./carazone -u Cmdenv -c Benign
./carazone -u Cmdenv -c DelayAttack
./carazone -u Cmdenv -c DriftAttack
./carazone -u Cmdenv -c SyncStarvation
```

The graphical environment is used for topology inspection and single-run tracing:

```
./carazone -u Qtenv -c Benign -r 0
```

Feature extraction. Extraction is invoked over a results directory:

```
python3 extract_features.py --results ../simulations/results --out features.csv
```

Window length, stride and simulation limit are command-line parameters with defaults of 0.25 s, 0.125 s and 10 s respectively. The default window is twice the gPTP synchronisation interval, so every window is expected to contain at least two correction events under normal operation. The stride is half the window length, giving 50% overlap between consecutive windows. Dependencies are `pandas`, `numpy`, and `opp_scavetool` from the OMNeT++ distribution on `PATH`.

Extraction pipeline. The script proceeds in five stages:

1. `run_scavetool()` invokes `opp_scavetool export -F CSV-R` over all `.vec` files in the results directory, producing one intermediate CSV. No scavetool filter expression is applied: match-expression syntax varies between tool versions and a rejected expression aborts the export, so the pipeline exports everything and filters in pandas. This trades a larger intermediate file for version robustness.

2. `load_export()` splits the export into a vector frame and a run-metadata frame. Vectors are filtered to names beginning `timeChanged`, excluding modules containing `underlyingClock` so that each node contributes its gPTP-corrected clock rather than its free-running oscillator. Run attributes and iteration variables are collected per run, which is how the configuration name and the victim zone are recovered.

3. `node_of()` and `zone_of()` map a module path to a node and a node to a zone. A regular expression matches the node types `gm`, `cvc`, `backbone`, `zone[`i`]` and `ecu[`j`]`. Zone controllers map to their own index; ECUs map by integer division by `ECUS_PER_ZONE`. Nodes with no zone (grandmaster, backbone and central vehicle compute) are excluded, since they are not federated clients.

4. `window_features()` computes eight statistics per node per window. The zero-event case returns an explicit record with `n_events = 0` rather than being discarded, because an empty window is itself the signature of synchronisation starvation. Offset is computed as `v - t` on the recorded series, giving error against the ideal timeline.

5. `build()` iterates runs, then zones, then windows. For each window it computes per-node features, isolates the zone controller's own vector as the `ctrl_` features, and aggregates mean and standard deviation across all nodes in the zone as the `zone_` features. Labelling is delegated to `label_for()`.

Labelling logic. `label_for(config, target_zone, zone, w_start)` returns the benign class unless three conditions hold simultaneously: the run configuration is an attack configuration; the zone under evaluation is the run's designated victim zone, read from run metadata; and the attack is active over the window, meaning either the window begins at or after `ATTACK_START` or the configuration is in `ALWAYS_ON_CONFIGS`. A bystander zone in an attack run is therefore labelled benign, which is what makes zone attribution a learnable target.

6.7 Algorithms and Technical Decisions

Windowing. Raw per-node clock series are segmented into fixed-size observation windows. The window size is a single global design parameter applied consistently across all

features so that one aligned feature vector is produced per window per zone.

Two-level feature construction. Each window yields both the zone controller's own statistics (`ctrl_`) and the distribution of those statistics across all nodes within the zone (`zone_mean_`, `zone_std_`). The controller is the node that would host the detector in deployment, so its own view is preserved separately; the within-zone spread captures whether a perturbation affects the controller alone or the whole zone.

Overlapping windows and split granularity. The stride is half the window length, so consecutive windows share half their samples. Adjacent windows are therefore strongly correlated, and a random window-level train/test split would leak information across the boundary. Splits are consequently formed at run and seed granularity (integrity rule R3).

Planned: cross-zone residual. The implemented features are all computed within a single zone. The design intent is that the discriminating feature is a residual *between* zones, taking a zone's controller offset minus the median across its siblings in the synchronisation tree, because that is the quantity a generic traffic-feature detector cannot construct. Computing it requires the observation window to be the outer loop so that all zones are available simultaneously, which is a restructuring of `build()` scheduled in Iteration 03.

Consequence for attack coverage. A sibling residual cancels common-mode perturbation by construction, and is therefore insensitive to any effect that moves the whole synchronisation tree together, including compromise at the grandmaster. This is a structural property rather than a tuning deficiency, and it bounds the detection claim to link-local timing integrity (Sections 2.7 and 7.6).

6.8 Tools, Frameworks and Technologies

Tool	Version	Purpose
OMNeT++	6.4.0	Discrete-event simulation kernel; Cmdenv for batch runs, Qtenv for inspection
INET [31]	4.7.0	TSN and gPTP protocol models, clock drift modelling
Python	3.x	Feature extraction, dataset construction
Flower	not yet adopted	Federated learning framework; selected for Iteration 03, not yet introduced into the build

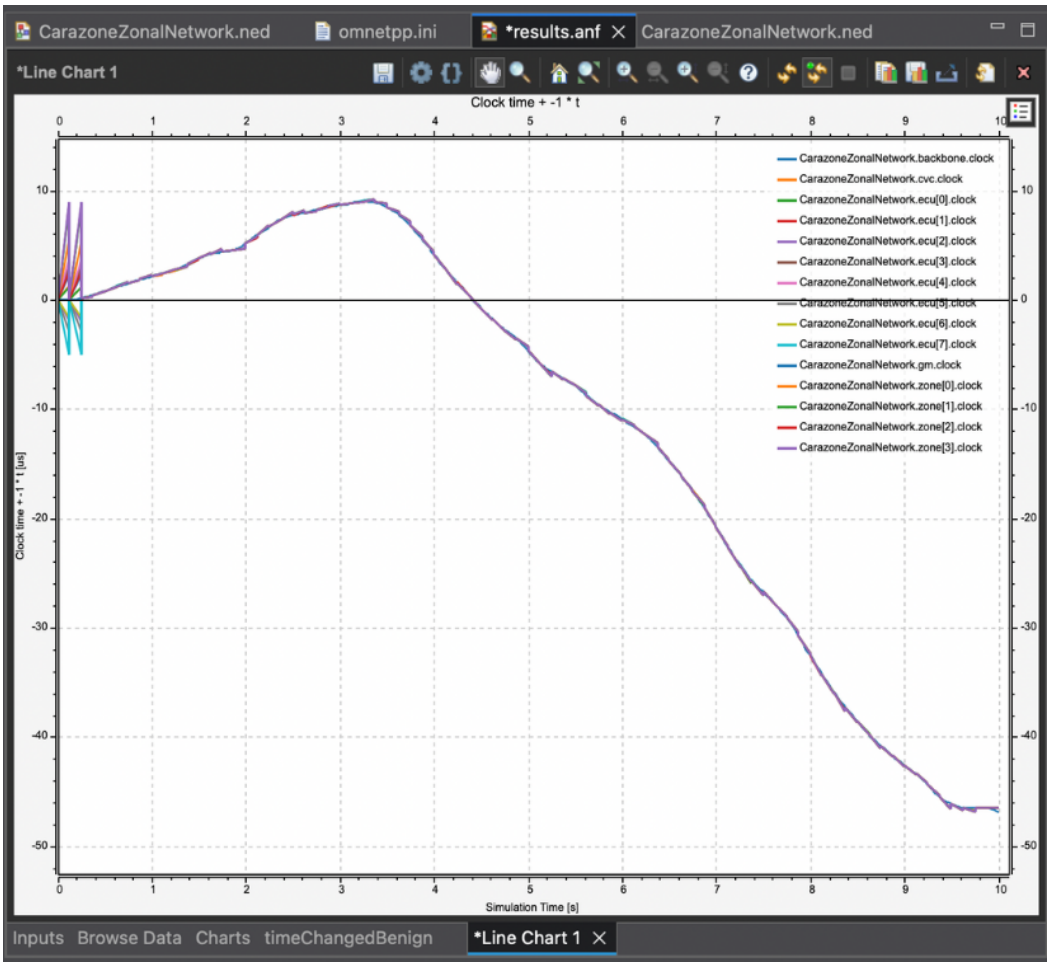


Figure 6.2: Clock offset against simulation time, benign run. All fourteen clocks follow a single common trajectory; the individual traces are not separable at this scale, which is the expected behaviour of a converged gPTP domain. The transient before approximately $t = 0.5$ s is the synchronisation servo converging from its initial state. That transient is the benign population discussed in Section 7.6: it produces larger offset excursions than the attack does, which is why a fixed threshold on offset alone cannot separate the two.

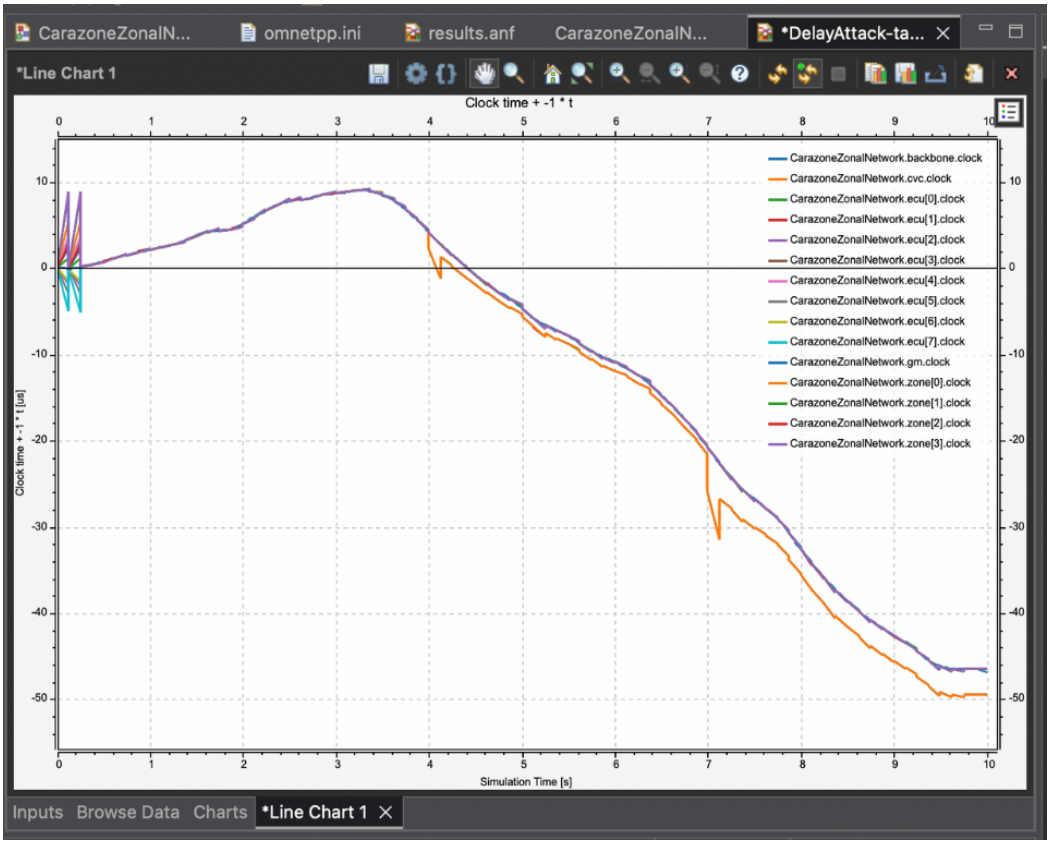


Figure 6.3: Clock offset against simulation time, delay attack on zone 0. The targeted zone controller (orange) separates from the common trajectory at $t = 4$ s, the attack onset, with a second visible step at $t = 7$ s; every other clock continues to track the group. The attack is visible only as a disagreement between one zone and its siblings, not as an absolute fault in the victim's own clock, which is the observation the cross-zone sibling residual is built to capture (Section 6.7).

6.9 Demonstration Evidence

Read together, Figures 6.2 and 6.3 are the empirical basis of the sibling residual feature. In the benign run every clock follows one trajectory. In the attacked run the targeted branch departs from that trajectory at the attack onset while the siblings continue to track one another. The quantity that changes between the two figures is not the victim's offset in absolute terms, which continues to follow the same broad shape, but its *disagreement* with the zones beside it. That is precisely the quantity the residual measures, and it is why a feature computed inside a single zone cannot see this attack while a cross-zone comparison can.

6.10 Attack Simulation Results on the Ring Topology

The results in Sections 6.9 and 7.6 were obtained on the four-zone tree. This section reports attack simulations on the five-zone ring reference topology of Section 5.6: five TSN switches, one gPTP grandmaster, and 25 endpoint ECUs together with the zonal controllers. Three attack families were run: constant oscillator drift, asymmetric link delay, and two attacks on an IEEE 802.1CB (FRER) protected stream. The aim was not yet detection accuracy but a narrower prior question: does each attack produce a measurable, separable signal that could serve as a feature?

6.10.1 Simulation pipeline and scenarios

All ring scenarios derive from a single generator (`gen_topology.py`) that emits the OMNeT++ topology description and the benign baseline configuration from editable tables of zones, links and traffic flows. A separate attack configuration file includes the baseline and layers each attack on as a selectable scenario, so an attack run differs from the baseline only by the injected fault. A launcher (`run_experiments.py`) executes the runs and writes a manifest recording, for every run, the scenario, the victim node and the attack window. Labelling for the extraction stage is therefore taken from the manifest at launch rather than reconstructed afterwards.

Benign traffic consists of five UDP flows, one representative per zone, all addressed to the central compute unit. Result recording was extended beyond delivery counts and the synchronised clock to include the uncorrected underlying oscillator and the gPTP correction signals `gmRateRatio`, `neighborRateRatio` and `pdelay`. Those correction signals turned out to be the informative ones (Section 6.10.3). The scenario catalogue is given in Table 6.2. Drift and FRER runs last two simulated seconds, with gPTP converging in the first second and the attack opening in the second, and the batch completes in well under a

minute.

Table 6.2: Ring scenario catalogue. All drift and FRER configurations run for two simulated seconds.

Configuration	Type	Mechanism	Status
`Baseline`	benign	no fault	run, validated
`Gptp_Drift_Deep`	gPTP	constant drift on `lidarRR`	run, analysed
`Gptp_Drift_Shallow`	gPTP	constant drift on `gnssReceiver`	run
`Gptp_Drift_Sweep`	gPTP	500, 2000, 5000, 10000 ppm (four runs)	run
`Gptp_Drift_Timed`	gPTP	drift introduced at $t = 1$ s	not runnable (Section 6.10.7)
`Frer_Enabled`	benign	redundant `brakeECU` → `centralCompute` stream	run, validated
`Frer_PathCut`	FRER	crash `swFR` for 0.5 s	run, analysed
`Frer_DuplicateFlood`	FRER	rogue node floods the protected port	run, analysed
`Frer_Plus_GptpDrift`	combined	drift on a FRER-path node	run

6.10.2 Benign baseline

The baseline completed to $t = 2$ s with no packet loss, and delivery at the central compute unit matched the configured flow rates exactly: 17,000 packets in total across the five flows (2000, 8000, 2000, 4000 and 1000 for the 1 ms, 250 μs, 1 ms, 500 μs and 2 ms flows respectively). gPTP converged, and the victim node's synchronised clock stayed within roughly 2 μs of the grandmaster for the whole run. That benign wander of about ± 2 μs is the noise floor against which the claims below are measured.

6.10.3 Constant drift attack

A constant oscillator drift was forced on `lidarRR`, the deepest node in the synchronisation tree. Read from the synchronised clock, the attack is almost invisible. At 500 ppm the clock differs from the baseline by only about 1.1 μs at $t = 2$ s, which is smaller than the benign wander of about ± 2 μs. A detector reading clock offset alone would not separate the two.

The attack is visible in the correction the servo applies to stay locked. Under a 2000 ppm drift, `gmRateRatio` moved from about 0.99999 in the baseline to about 0.99800 and stayed there for the whole run (Table 6.3). That is a displacement of roughly 2000 ppm

against benign fluctuations of tens of ppm, so about two orders of magnitude above the benign variation. `neighborRateRatio` shifts similarly, but it updates only once per peer-delay measurement and yields about two samples per run, so it is usable only as a coarse per-window statistic; `gmRateRatio` is the dense signal.

Table 6.3: Constant drift on `lidarRR`: the servo's correction records the attack that the corrected clock hides.

Signal (at $t \approx 2$ s)	**Baseline**	**Attack**
Synchronised clock offset (500 ppm)	benign wander $\approx \pm 2\ \mu$s	shift of $\approx 1.1\ \mu$s
`gmRateRatio` (2000 ppm)	≈ 0.99999	≈ 0.99800, flat
`neighborRateRatio` (2000 ppm)	≈ 1.00006	≈ 0.99807

A severity sweep at 500, 2000, 5000 and 10000 ppm showed the `gmRateRatio` displacement increasing monotonically with the injected drift, approximately equal to the drift itself. This gives a graded severity axis for training, and shows that the primary feature responds proportionally rather than only above a threshold.

The mechanism is straightforward. A servo that must continuously compensate a drifting oscillator records that effort in its rate ratio, so the attack appears in the correction even when the corrected output looks clean. This is why clock offset is insufficient as a drift feature and rate ratio is necessary.

6.10.4 Asymmetric delay attacks

Asymmetric delay was injected on the link at `swFL`. Table 6.4 gives the clock offset of three nodes before the attack and during the 7–10 s window.

Table 6.4: Asymmetric delay on `swFL`: offset shift at the victim, a downstream node and a control node.

Node	**Role**	**Before (μs)**	**During, 7–10 s (μs)**	**Shift (μs)**
`swFL`	victim	0.01	2.72	2.71
`zcFL`	downstream	0.006	2.718	2.711
`swFR`	control (opposite side)	0.012	0.011	−0.001

The victim shifts by 2.71 μs. The zonal controller behind it inherits almost the same shift, 2.711 μs, because gPTP distributes time down a chain and a corrupted parent hands its error to its children. The control node on the opposite side of the ring does not move. This is the spatial pattern on which the sibling-residual feature rests: the attack appears as a disagreement between one branch and its siblings rather than as a network-wide change.

The peer-delay measurements show where the error enters. The peer delay measured at `swFL` rose from about 10 ns before the attack to about 3.4 μs on average during it, while the peer delay measured at `zcFL` stayed at zero. The error is therefore injected on one link and propagates by inheritance; the downstream node is not mismeasuring its own link. This makes the attack attributable to a link, which is what success criterion SC-3 requires of an alert.

6.10.5 FRER attacks

With FRER enabled, the full 802.1CB pipeline (stream identification, sequence encoding and decoding, and the stream-relay merger) instantiated at the split point `swFL` and the merge point `swC`. The protected stream is the 1 ms `brakeECU` $\rightarrow$ `centralCompute` flow, replicated over two disjoint ring paths. In the benign FRER run the merger at `swC` received 24,060 frames, forwarded 22,060 and dropped 2000, each dropped frame being 1024 B, which is the protected stream. The arithmetic closes, and the sink received all 2000 frames. The merger's dropped-frame count is therefore a direct measure of redundancy health, with a benign value equal to the protected stream's frame count.

Table 6.5: FRER signals under the two attacks. Each attack is visible in a different counter.

Scenario	**Duplicates eliminated at `swC`**	**Delivery at sink**	**Visible in**
Benign (`Frer_Enabled`)	2000	2000	reference
Path cut	**1500**	2000	FRER mechanism
Port flood	2000	**12000**	application rate

Path cut. `swFR` was crashed at $t = 1$ s and restarted at $t = 1.5$ s, taking one redundant path down for half a second. Delivery was uninterrupted, because FRER masked the failure, so a packet-loss monitor would see nothing. The eliminated-duplicate count fell by exactly 500. That matches the outage: 0.5 s at 1000 frames per second leaves 500 frames with no duplicate to eliminate. The failure is invisible in the service metric and exact to the frame in the protocol counter.

Port flood. A rogue node, `radarFL`, sent to the protected port at five times the stream rate. The elimination counter did not change: the 2000 genuine duplicates were still the only frames dropped. The flood frames passed through to the application, so delivery at the sink rose from 2000 to 12000, which is the original 2000 plus the 10,000 flood frames. The cause is that 802.1CB elimination keys on stream identity and sequence number, not on destination port, so a same-port flood never matches the protected stream. The attack appears as application-layer overload with the FRER mechanism untouched. It also shows

why a true sequence-number replay, which would deceive elimination, requires tampering inside the sequence-encoding module and so cannot be produced by configuration alone.

6.10.6 Detection signals identified

Table 6.6 collects the signals. In every case the naive metric (synchronised clock time, packet loss) is blind or ambiguous, and the protocol-mechanism metric (gPTP rate ratio, spatial offset pattern, FRER elimination count) is not.

Table 6.6: Candidate detection features identified by the ring attack simulations.

Attack	**Primary feature**	**Benign**	**Under attack**
Constant drift	`gmRateRatio` per node	≈ 1.0	persistent offset of about the drift
Asymmetric delay	offset shift of a node relative to its siblings; `pdelay` on the victim link	bounded; ≈ 10 ns	victim and descendants shift together; $\approx 3.4\ \mu$s on the link
FRER path cut	eliminations at `swC`	$\approx$ stream frame count	falls while a path is down
FRER port flood	delivery count at sink	matches flow rate	rises

6.10.7 Limitations of these results

These results are single runs. Repeated seeds are needed to establish variance before any model is trained on them. The attacks are configuration-level, acting on the oscillator, the link and the traffic, rather than protocol-level tampering with gPTP messages or 802.1CB sequence numbers, which remains the planned extension. A constant drift is the easiest case for the servo to compensate, so the rate-ratio result should not be read as covering time-varying drift. That case cannot be produced by configuration, because INET's `driftRate` parameter is not marked mutable and cannot be changed mid-run, which is why `Gptp_Drift_Timed` fails at the moment the change is attempted. This is the same constraint described in Section 7.6. The finding here is narrower than that section's negative result and does not contradict it: a runtime drift attack remains unrealisable under this framework, but a drift present from $t = 0$ is detectable through the servo's correction signal even though its clock offset is not.

6.11 Iteration Outcome

Iterations 01 and 02 met their exit criteria. The simulation produces repeatable, seeded attack effects; the extraction pipeline produces a labelled dataset. One defect (DEF-01) and one design limitation (zone symmetry) were identified and are scheduled for correction before Iteration 03 modelling work begins.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

Testing at this stage targets the data generation path rather than the detection model, on the principle that model results obtained from an unverified dataset are uninterpretable. Three levels are applied: component checks on extraction logic, integration checks on the simulation-to-dataset path, and empirical validation that injected attacks produce the theoretically expected signature.

7.2 Test Levels and Techniques

- **Component.** Assertions within the extraction pipeline on window counts, absence of missing values, and per-zone label distribution.
- **Integration.** End-to-end execution from configuration to dataset, verifying row counts and schema conformance.
- **Empirical validation.** Comparison of observed attack signatures against the expected theoretical effect, across multiple seeds.
- **Reproducibility.** Repeated execution with identical seeds, comparing generated structure and labels.

7.3 Test Cases

Table 7.1: Test cases executed at the mid-report stage

ID	Objective	Expected	Result
TC-01	Topology instantiates with all declared nodes	All nodes present and linked	Pass
TC-02	gPTP synchronises all time-aware nodes	Node clocks track a common trajectory under benign operation	Pass
TC-03	Seeded reproducibility	Identical seed reproduces identical structure and labels	Pass
TC-04	Delay attack produces measurable deviation	Targeted branch diverges from siblings during the attack interval	Pass
TC-05	Drift attack produces zone-localisable deviation	Targeted zone diverges from siblings	**Not realisable; see Section 7.6**
TC-06	Extraction produces expected window count	Windows = runs × zones × windows-per-run = $15 \times 4 \times 79$	Pass (4,740)
TC-07	Zero-event windows are retained, not dropped	Window with no clock event yields a record with `n_events`$= 0$	Pass
TC-08	Attack labels attributed to victim zone only	In an attack run targeting zone z, only zone z carries the attack label	Pass (see Section 7.4)
TC-09	Dataset conforms to the documented schema	Column names, order and types match Table 5.6	Pass (after schema amendment; see Section 7.7)

7.4 Test Results

Fifteen seeded runs produced 225 recorded vectors, from which the extraction pipeline generated 4,740 labelled feature windows.

Table 7.2: Label distribution by zone

Zone	Benign	Attack	Total
Zone 0	1,044	141	1,185

Zone	Benign	Attack	Total
Zone 1	1,044	141	1,185
Zone 2	1,044	141	1,185
Zone 3	1,044	141	1,185
Total	**4,176**	**564**	**4,740**

Derivation of the distribution. With a window of 0.25 s, a stride of 0.125 s and a simulation limit of 10 s, each run yields 79 windows per zone. The observed 1,185 windows per zone therefore correspond to exactly 15 runs. Attacks begin at $t = 4.0$ s, so 47 of the 79 windows per run fall within the attack interval. The observed 141 attack-labelled windows per zone correspond to 3×47, that is, each zone is the designated victim in exactly three runs. The run set is accordingly 3 benign runs and 12 delay-attack runs, covering four victim zones at three seeds each, giving $12 \times 47 = 564$ attack windows in total.

Victim rotation. Rotating the victim zone across runs is a deliberate design choice. Had a single zone been targeted throughout, zone identity would correlate perfectly with the label and any classifier could reach high accuracy by learning zone index alone. Because every zone appears as victim in three runs and as bystander in nine, zone identity carries no label information and attribution must be learned from the timing features.

Class imbalance. The benign-to-attack ratio is approximately 7.4:1. Accuracy is therefore excluded as a reported measure; evaluation uses per-class precision, recall and F1, macro-F1, and detection rate at a fixed false-positive rate.

7.5 Acceptance Testing / Validation

Acceptance checks defined in Section 4.6 have been exercised for FR-01 through FR-09. Checks for FR-10 to FR-15 and NFR-08 require the federation and monitoring-interference work scheduled for Iteration 04 and are reported as planned in Section 9.5.

Success criterion SC-2, that protocol-native features outperform features computable without knowledge of network structure, has been tested centrally ahead of schedule, because it is the premise on which the federated work depends and a null result would have required repositioning the project. The experiment is a feature ablation with identical models and identical splits across three feature sets:

- **within_zone** (24 features): all `ctrl_` and `zone_` statistics. These describe one zone's clock behaviour in isolation and require no knowledge of the synchronisation tree. This is the control arm.

- **residual** (2 features): the cross-zone sibling residuals only.
- **combined** (26 features): both.

Splits are formed by holding out whole runs, stratified so that both benign and attack runs appear on each side, and averaged over five resplits. Window-level splitting is not used: the extraction stride is half the window length, so adjacent windows share half their samples and a random split would place near-duplicates on both sides.

Table 7.3: Feature ablation, macro-F1 averaged over five run-level splits

Feature set	**Model**	**Macro-F1**	**Attack recall**	**Latency (windows)**
Majority-class floor	n/a	0.468	n/a	n/a
within_zone	logistic reg.	0.500	0.419	11.6
within_zone	MLP	0.481	0.013	25.0
within_zone	random forest	0.536	0.259	12.7
residual	logistic reg.	0.951	1.000	2.0
residual	MLP	0.968	0.993	2.0
residual	random forest	**0.984**	0.994	2.5
combined	logistic reg.	0.975	0.997	2.0
combined	MLP	0.966	0.919	2.0
combined	random forest	0.969	0.930	2.0

Result 1: within-zone features do not detect this attack. All three within-zone arms fall between 0.481 and 0.536 macro-F1 against a majority-class floor of 0.468. The MLP arm attains 0.013 attack recall, that is, it classifies almost every window as benign. Features describing a single zone's clock in isolation carry essentially no information about this attack, because the attack does not make the victim's clock behave abnormally in absolute terms; it makes the victim disagree with its siblings.

Result 2: two cross-zone features are sufficient. The residual arm reaches 0.951–0.984 macro-F1 using two columns. The gain over the control arm is $+0.475$, $+0.485$ and $+0.433$ macro-F1 for logistic regression, MLP and random forest respectively. SC-2 is therefore met, and met by a margin that does not depend on model choice.

Result 3: the larger feature set is not the better one. For the MLP and random forest, the residual arm alone outperforms the combined arm (0.984 against 0.969 for random forest). The 24 within-zone features dilute rather than augment the signal. This is relevant to NFR-05: a detector intended to run on a resource-constrained zone controller benefits from a deliberately small feature set, and the evidence here is that the small set is also the more accurate one.

Result 4: both residual features are necessary. That logistic regression, a linear decision boundary, reaches 0.951 macro-F1 on two features implies that the classes are linearly separable in the two-dimensional residual space. They are not separable in the offset dimension alone, as Section 7.6 shows. The drift-slope residual is therefore load-bearing despite appearing weak when summarised univariately, and the mechanism by which it contributes is recorded in Section 7.6.

Detection latency. With a confirmation requirement of three consecutive anomalous windows, detection fires two windows after attack onset for every residual-based arm. At a stride of 0.125 s and a synchronisation interval of 0.125 s, that is two synchronisation intervals, which is the minimum achievable under a three-window confirmation rule. The within-zone arms require 11.6 to 25.0 windows, the upper value indicating that detection frequently does not occur at all within the run.

Error structure. For the combined random-forest arm the confusion matrix over a held-out split is $\left(\begin{smallmatrix} 1389 & 3 \\ 0 & 188 \end{smallmatrix}\right)$ (rows true, columns predicted): no attack window is missed, and three benign windows are misclassified. Logistic regression shows the same structure with nine false positives and no false negatives. The error is therefore one-sided towards false alarm rather than missed detection, which is the preferable direction for a safety-relevant monitor, though it makes the false-alarm rate the quantity requiring most attention at deployment scale (Section 9.5).

7.6 Edge and Failure Cases

Negative result: oscillator drift manipulation is not realisable as a runtime attack under the current simulation framework. An attack was designed in which the scenario manager would alter the local oscillator drift rate of a victim node partway through a run, producing a zone-localised divergence from the remaining zones.

The attack could not be implemented in that form. The clock module's drift-rate parameter is not declared mutable in the INET version in use, so the scenario manager cannot modify it at simulation time. The implementation therefore applies the manipulated drift rate from $t = 0$, which is why the `DriftAttack` configuration appears in the extraction script's `ALWAYS_ON_CONFIGS` set rather than being gated on the attack start time.

Applied from $t = 0$, the manipulation is not an attack in any meaningful sense: it is a node whose oscillator runs at a different constant rate for the whole simulation, which, in the clock-offset dimension measured here, is indistinguishable from legitimate component variation and which the gPTP servo is specifically designed to compensate. The observed network-wide behaviour is the protocol operating correctly, not a detection failure.

Three consequences are carried forward. First, the finding is a statement about attack realisability under this framework, not about the detectability of drift attacks in general; the stronger claim is not supported by this experiment and is not made. Second, drift is removed as a committed target for the cross-zone offset residual, reducing the committed attack set to delay injection and synchronisation starvation; a separate candidate signal for constant drift is described below. Third, if a runtime drift attack is required in FYP-2, it must be realised by a different mechanism, for example by subclassing the clock module to expose a mutable rate, or by post-processing the recorded trace, and the cost of that work is weighed against the alternative of deepening the evaluation of the two realisable attack classes.

Update from the ring topology. Constant drift applied from $t = 0$ was later re-run on the five-zone ring with the gPTP correction signals recorded (Section 6.10). The clock offset still hides it, but `gmRateRatio` moves from about 0.99999 to about 0.99800 at 2000 ppm, so the signal that is absent from the offset dimension is present in the rate-ratio dimension. This does not change the realisability finding above, since a runtime drift attack remains unrealisable, and it leaves the committed attack set unchanged pending extraction of rate-ratio features.

Negative result: no fixed threshold separates attack from benign behaviour. Two tuned thresholds on the absolute sibling offset residual were evaluated as non-learned baselines. The first was set to the largest residual observed on benign training windows, which by construction admits all benign training behaviour. The second was set to the 99.5th percentile of that distribution, discarding the extreme tail. Both attained zero recall on held-out data: every attack window fell below both thresholds.

The cause is the gPTP convergence transient. Table 7.4 separates benign windows by simulation time.

Table 7.4: Absolute sibling offset residual by population

Population	**Median (s)**	**Maximum (s)**
Benign, $t < 1$ s (convergence)	1.00×10^{-8}	1.51×10^{-5}
Benign, $t \geq 1$ s (steady state)	4.83×10^{-9}	1.81×10^{-6}
Delay attack	2.19×10^{-6}	5.66×10^{-6}

Before the clock servos lock, zones disagree with one another by more than any injected

attack subsequently moves them: the convergence maximum exceeds the largest attack residual by a factor of 2.7. Those early windows are legitimately benign, so any threshold admitting all benign behaviour must sit above the startup excursion and is consequently blind to the attack in steady state.

The percentile variant fails for a quantitative reason worth stating. Windows beginning before $t = 1\,\mathrm{s}$ constitute approximately 11% of all benign windows, so any percentile above roughly the 89th remains inside the convergence population. The failure is therefore not an artefact of a poorly chosen cut-off: no percentile-based threshold succeeds while the convergence transient is part of the benign class.

This is a stronger justification for the learned detector than the ablation alone provides. The ablation establishes which features carry the signal; this result establishes that the signal is not separable by a single decision boundary on the offset residual, because the benign class is not unimodal; it is a mixture of a steady-state population and a convergence population, and the attack lies between them in magnitude.

Why two residual features are required. The second residual feature, computed on the drift slope rather than the mean offset, is a weak discriminator when measured alone: the ratio of its median under attack to its median under benign operation is approximately 2.0, against 438 for the offset residual. It is nevertheless not redundant. The residual feature set contains only these two columns, and logistic regression, a linear decision boundary, attains 0.951 macro-F1 on them (Table 7.3). A linear boundary can only achieve this if the three populations are linearly separable in the two-dimensional residual space, which they are not in the offset dimension alone.

The mechanism is that the two populations the offset residual confuses are distinguished by the slope. During convergence the servos are actively correcting, so successive offsets change rapidly and the slope residual is large. The delay attack introduces a steady asymmetry, so the victim's offset departs from its siblings along a near-constant gradient and the slope residual remains small. The slope residual is therefore not a second detector of the attack; it is the feature that excludes the benign convergence population from the region of offset-residual space that the attack occupies. This is the reason a univariate summary understates its contribution, and it is recorded here because the measurement that reveals it, a linear model succeeding on two features, is easily mistaken for an unimportant result.

The practical consequence for FYP-2 is that the attack-intensity sweep (Section 9.5) must report the benign envelope, separated into convergence and steady-state populations, alongside detection rate rather than reporting detection rate alone.

Pending: grandmaster-level perturbation. A separate structural limitation follows from the planned sibling-residual feature rather than from any implemented result. A residual between siblings cancels common-mode perturbation by construction, so it can-

not detect an effect that moves the entire synchronisation tree together, including compromise at the time source itself. This bounds the detection claim to link-local timing integrity. Detecting time-source compromise would require an independent reference such as local oscillator holdover comparison or a second synchronisation domain, and is out of scope.

Pending edge cases. Attack intensities approaching the benign drift envelope, unseen seeds, and coordinator or model unavailability are scheduled for Iteration 04 (Section 9.5).

7.7 Defects and Corrective Actions

DEF-04. Schema drift between the documented column contract and the emitted dataset. Test TC-09 compared the columns of `features.csv` against the contract in Table 5.6 and found three columns present in the data but absent from the contract: `target_zone`, and the two `sibling_n_{f}` counts emitted alongside each residual. *Consequence:* a reader implementing against the documented contract would not have anticipated them, and `target_zone` in particular is equivalent to the label, so an implementer unaware of it could leak ground truth into a model. *Recovery:* the contract in Table 5.6 was corrected to document all three, and the leakage risk is additionally guarded by an automated test asserting that no identifier column reaches any feature set (Appendix **??**). Status: **closed**.

Table 7.5: Defect register

ID	Description	Impact	Corrective action
DEF-01	The feature set contained no cross-zone comparison; all features were computed within a single zone.	The features were not structurally exclusive to a deterministic network, so SC-2 could not be tested.	`build()` restructured to iterate window-then-zone, and two sibling residual features emitted. Verified by the ablation in Section 7.5: within-zone features reach 0.48–0.54 macro-F1, residual features 0.95–0.98. Status: **closed**.

ID	Description	Impact	Corrective action
DEF-02	All four zones of the Iteration 1 topology carry identical traffic profiles and produce identical window counts.	Federated, centralized and local-only training would converge to equivalent models, making the federated comparison (SC-4) null by construction.	The five-zone reference topology (Section 5.6) introduces structural asymmetry: three to seven ECUs per zone, distinct traffic classes by zone function, and gPTP delay that grows with hop distance from the grandmaster. Status: **partially addressed**. Designed but not measured; closes when per-zone divergence is quantified and differentiated traffic profiles are running.
DEF-03	The attack start time is hard-coded in the extraction script as a module constant and must be kept consistent with the simulation configuration by hand.	A change to the attack schedule in the simulation that is not mirrored in the extraction script would mislabel the dataset silently.	Read the attack start time from run metadata rather than from a constant, and add an assertion that fails extraction when the two disagree. Status: open, low effort.

7.8 Current Validation Status

The data generation path is validated for one attack class and found defective in one respect. The detection and federation paths are unvalidated because they are unimplemented. No detection performance figures are claimed at this stage, and none should be inferred from the dataset statistics reported above. The attack simulations on the five-zone ring (Section 6.10) are single-run observations of candidate signals; they have not been extracted into the dataset, and no detection performance is claimed from them.

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

Zonal TSN topology with gPTP synchronisation; attack injection harness with three instrumented configurations; 15 seeded simulation runs producing 225 recorded vectors; feature extraction pipeline producing 4,740 labelled windows; empirical validation of delay-attack signature; identification and characterisation of one negative result and two defects; literature review and departmental-repository gap comparison; operator-facing alert surface as a presentation mock-up.

8.2 Work Remaining

Correction of DEF-01 and DEF-02; centralized detection model and the protocol-native-versus-generic-feature comparison; federated training pipeline with local-only, centralized and FedAvg arms; zone attribution evaluation; detection latency measurement; monitoring-interference measurement (NFR-08); alert surface driven by real model output; rule-based fallback.

8.3 Individual Responsibilities

Responsibilities follow the allocation in Table 3.2 (Section 3.6). The split is by area rather than by task, so that each member owns a coherent part of the system end to end and can explain it independently. Adeena Zaib owns the simulation and data-generation path: the OMNeT++/INET topology, the attack scenarios, and the per-zone dataset packaging that

feeds the federated clients. Khaula Bilqees owns the TSN signal instrumentation, the labelling scheme, and the federated client and server implementation.

8.4 Contribution of Each Team Member

The artefacts produced in this reporting period are listed below against their owner. Each is verifiable against the commit history in Appendix **??** rather than asserted here.

Khaula Bilqees. The OMNeT++/INET zonal topology and its scenario configurations, committed as `8b765f6`; and the subsequent topology and configuration revisions in `31cb803` and `f009ac6`.

Adeena Zaib. The initial proof of concept (`0506e63`); the feature extraction pipeline including the window construction, the labelling rule and the cross-zone sibling residual (`856beba`); and the continuous integration workflow with its automated test suite (`ed4b0f6`, `2168990`).

The commit record shows seven commits, four from the first author and three from the second, with both members committing under their own accounts throughout.

Divergence from the planned allocation. Table 3.2 in Section 3.6 allocates the topology to the first author and the signal instrumentation to the second. In practice these two areas were exchanged during Iteration 1: the topology was implemented by the second author and the extraction pipeline by the first. The allocation of the remaining areas is unchanged, and the federated client and server implementation scheduled for Iteration 03 remains with the second author. This is recorded rather than quietly corrected, because the commit history is the evidence for this section and it should agree with the text.

8.5 Repository and Version Control Evidence

The project is held in a Git repository with both members committing under their own accounts. Continuous integration is configured through GitHub Actions: a workflow runs on every push and executes the automated test suite described in Section 7.2, so a change that breaks the feature extraction or training pipeline is caught at the point it is pushed rather than at the next manual run.

The repository is at `https://github.com/AdeenaZaib/VROOM`. The commit log and the per-member commit counts are reproduced in Appendix **??**.

Two properties of this history are worth stating explicitly, because they are what the evidence is for. First, the pushes are distributed across both members: runs #1 and #2 were pushed by the first member and runs #3 and #4 by the second, so the contribution record

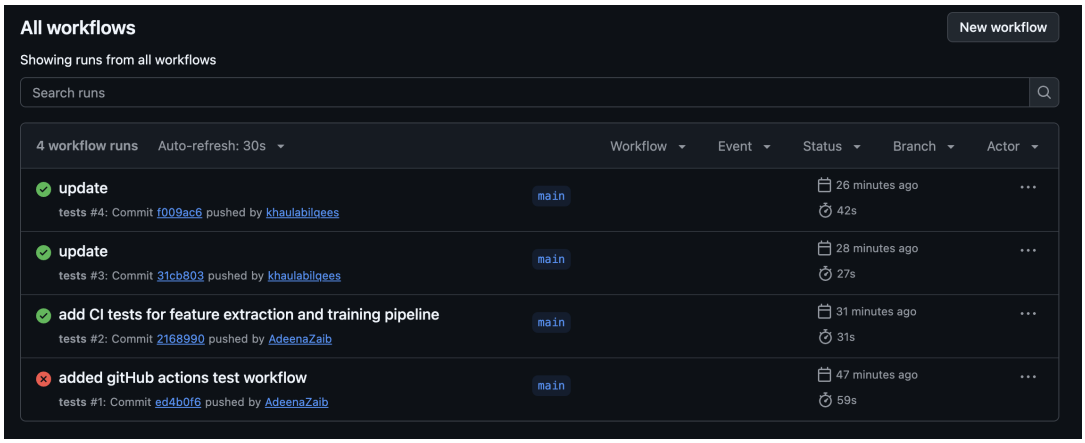


Figure 8.1: GitHub Actions run history. Each run is tied to a commit and to the member who pushed it, which is the per-member development evidence this section reports. Run #1 failed and the following runs pass, which records a real correction rather than a repository created immediately before submission.

is visible in the tooling rather than asserted in this document. Second, the first run failed and subsequent runs pass. That sequence is retained deliberately: it shows the pipeline being established and corrected during the reporting period.

8.6 Milestones and Commit Evidence

The automated suite is itself the milestone evidence for modules M2 and M3. It contains fourteen tests, of which three are dataset invariants that run only when `features.csv` is present and are therefore exercising the real extracted dataset rather than synthetic fixtures. Three of those tests encode the corrections recorded as defects in Section 7.7: that attack labels are confined to the target zone (DEF-01), that the cross-zone residual is not identically zero, and that the victim zone rotates across runs so that the detector cannot memorise a single zone.

Table 8.1: Milestones against the commits that completed them

Commit	Milestone	Module and evidence
`0506e63`	Proof of concept	Pre-proposal demonstrator
`8b765f6`	M1 complete: zonal topology simulated	`CarazoneZonalNetwork.ned`; Figure 6.1

Commit	Milestone	Module and evidence
`856beba`	M3 complete: feature extraction, including the cross-zone residual	`extract_features.py`; ablation in Section 7.5
`ed4b0f6`	Continuous integration established	First workflow run; failed, and corrected in the next commit
`2168990`	Automated test suite covering M2 and M3	Fourteen tests; three encode the DEF-01 correction (Appendix **??**)
`31cb803`, `f009ac6`	Topology and configuration revisions	Passing workflow runs #3 and #4

8.7 Tools and Tool-Assisted Development

Table 8.2: Tool and AI-assistance disclosure

Tool	Used for	Boundary
OMNeT++ / INET	Simulation framework	Standard open-source components used as published; the topology, scenario configurations and attack injection logic are the team's own work
Generative AI assistants	Literature search support, document drafting, code scaffolding, report structuring and review	Core detection model design, federated architecture decisions, attack implementation, simulation configuration, experimental results and their interpretation are the team's own work. Generated text has been reviewed, verified against primary sources and rewritten by the authors. No reported result is AI generated.

8.8 External Components and Dependencies

Component	Version	Licence	Use
OMNeT++	6.4.0	Academic Public License	Simulation kernel
INET [31]	4.7.0	LGPL	TSN, gPTP, clock models
Python	3.x	PSF	Extraction, training
Flower	not yet adopted	Apache 2.0	Federated learning framework; selected for Iteration 03, not yet introduced into the build

8.9 Risks and Mitigation

Table 8.4: Risk register

Risk	L / I	Mitigation	Trigger / fallback
Protocol-native features do not outperform generic traffic features	Med / High	Run the controlled comparison centrally and early, before federated infrastructure is built	If no advantage is demonstrated, report the negative result and reposition the contribution around federated zone-level detection and attribution
Required internal protocol state is not exposed by the simulation framework	Med / High	Verify observability per feature before committing the feature	Substitute an observable proxy or instrument the module explicitly
FedAvg does not outperform local-only training under non-IID zones	Med / Med	Introduce genuine zone heterogeneity; normalise features; report worst-client performance	Report as a legitimate finding; FedProx retained as an extension, not a rescue
Feature leakage through adjacent-window splitting	Med / High	Split by run and seed, never by window (integrity rule R3)	Rebuild the dataset and record the change

Risk	L / I	Mitigation	Trigger / fallback
Scope expansion through additional attacks or models	High / High	Protect committed scope; extensions gated on completion of the core path	Drop schedule-window detection, FedProx and conformal calibration first, in that order; reduce FRER to a single redundant stream pair before dropping it
Remaining implementation compressed into FYP-2	High / High	Sequence the decision gate first so that a null result is found early	Reduce to a single attack class with full evaluation rather than several partially evaluated

8.10 Security and Privacy Considerations

All data is synthetic. No personal, vehicle or OEM data is processed. The federated design transmits model parameters only; raw feature records never leave the client (NFR-07), which is verified by the acceptance check on FR-13. Attack implementations are confined to the simulation environment. Secure aggregation, differential privacy and model-poisoning resistance are outside committed scope and are stated as limitations rather than claimed properties.

8.11 Current Limitations

1. Detection results are centralized only; no federated result is claimed.
2. Zones are currently symmetric, so federated behaviour cannot yet be assessed.
3. The injected attack is large relative to the benign steady-state envelope; the intensity at which detection degrades has not been established.
4. Oscillator drift manipulation is not realisable as a runtime attack under the current framework, reducing the committed attack set to two classes. A constant drift is, however, visible in the servo's rate ratio on the ring (Section 6.10).
5. Time-aware shaping is not configured, so schedule-conformance features and deadline-miss consequences are unavailable.
6. The approach is insensitive to grandmaster-level compromise by construction.

7. The static-environment assumption (A1) excludes networks that reconfigure at runtime.

8. Evaluation is simulation-only; attacks are synthetic and no hardware validation is performed.

9. The alert surface is a presentation mock-up and is not yet driven by model output.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

Objectives O4–O6: detection model, federated training and comparative evaluation, and attributed alerting driven by real model output.

9.2 Remaining Modules and Features

M4 Federated Detection Model; M5 Zone Participation and Aggregation; M6 Alerting and Presentation; the redundant-topology and IEEE 802.1CB FRER extension to M2 (topology and attack injection) and M3 (feature extraction); plus corrective work on M2 and M3 (DEF-01, DEF-02).

9.3 Remaining Iterations

Iteration 03: Detection and decision gate. Complete. The cross-zone sibling residual was added, a centralized detector trained, and the feature ablation completed. The decision gate is passed: cross-zone features exceed within-zone features by $+0.43$ to $+0.49$ macro-F1 across three models, so the federated work proceeds as planned.

Iteration 03b: Dataset depth. Extract the `SyncStarvation` configuration to give a genuine multiclass problem. Sweep the injected attack intensity downwards to establish the detection boundary against the benign envelope. Introduce differentiated per-zone traffic profiles (DEF-02).

Iteration 03c: Redundant topology and FRER. The topology and the first half of the FRER work are done. The five-zone ring (Section 5.6) is generated, passes its struc-

tural checks including the single-link-cut partition test that establishes disjoint paths, and has been run. The benign baseline completed without packet loss and with gPTP converged; IEEE 802.1CB replication and elimination were configured for the protected brake stream and confirmed; and two stream-redundancy attacks, a path cut and a same-port flood, were simulated and analysed (Section 6.10). What remains is to record the deadline and link-utilisation checks among the four behavioural criteria of Section 5.6.6, and to implement a protocol-level stream-redundancy attack: selective suppression or delay of one member of a replicated stream in a single target zone, or sequence-number replay, neither of which configuration alone can produce. Add the FRER feature family to `extract_features.py`, namely inter-member arrival skew, sequence-number deviation between redundant members, and per-member inter-arrival cadence, together with their cross-zone sibling residuals, computed by the same window-outer, zone-inner construction already used for the gPTP features. Exit criterion: the FRER residual features reach macro-F1 > 0.90 on the redundancy attack class while the corresponding within-zone features remain at chance, which would show the residual principle to be a property of the zonal architecture rather than of the synchronisation plane alone. This iteration is independent of Iteration 03b and may run in parallel, since it touches the topology and the feature extractor rather than the detector. The gPTP attack configurations have been migrated to the five-zone ring and run there (drift and asymmetric delay, Section 6.10). Re-establishing the Chapter 7 results on the reference topology, as a labelled dataset and a trained detector, remains, and is a prerequisite for both.

Iteration 04: Federation and evaluation. Federated client/server implementation; local-only, centralized-pooled and FedAvg arms on identical splits; zone attribution evaluation; detection latency; alert surface driven by model output; rule-based fallback; monitoring-interference measurement.

9.4 Implementation Plan

Iteration 03 proceeds in strict order: labelling correction, heterogeneity, third attack class, control arm, comparison. The ordering is deliberate, because each step is a precondition for interpreting the next. Iteration 03c runs in parallel on the topology and extraction path: redundant links, 802.1CB sequence generation and recovery, redundancy attack injection, then the FRER feature family and its residuals. Iteration 04 proceeds in order: local-only baseline, centralized upper bound, FedAvg, then the remaining evaluation dimensions.

9.5 Testing and Validation Plan

Evaluation dimension	Metrics
Detection quality	Per-class precision, recall, F1; macro-F1; confusion matrix; detection rate at fixed false-positive rate
Feature-set comparison	Macro-F1 of protocol-native versus generic traffic features, identical model and splits, reported across seeds
Operational detection	Detection latency in synchronisation intervals; false alarms per vehicle-hour
Attribution	Zone attribution top-1 accuracy, reported separately from detection
Federated behaviour	Worst-client macro-F1; centralized-to-FedAvg gap; convergence rounds; update volume per round
Baselines	Fixed threshold on raw offset; CUSUM on sibling residual; LOF (for comparability with published gPTP work)
TSN behaviour	Protected-stream mean and 99th-percentile latency, monitoring enabled versus disabled (NFR-08)
Reproducibility	Results across fixed seeds, held-out attack episodes and held-out configurations

Accuracy is excluded as a headline measure throughout, owing to the class imbalance reported in Section 7.4. Train/test separation is performed at run and seed granularity (integrity rule R3).

9.6 Deployment Plan

The deliverable is a reproducible research and validation environment: pinned dependencies, documented commands, sample configurations, seeds, and exported results, executable by a non-author on a clean environment.

Two consumers of detector output are defined. Within the vehicle, the integrity verdict (class, affected branch, confidence) is the decision input that fault-tolerant timing mechanisms require in order to fail over or enter holdover; the relevant standardised machinery provides the failover mechanism but leaves the domain-quality decision unspecified. Off-vehicle, attributed alerts populate the vSOC incident queue, where per-vehicle anomalies are triaged against fleet-wide correlation.

9.7 Tentative Timeline

The project proposal was defended on 11 September 2026 and this mid evaluation falls in the week of 13 October 2026. The final FYP-1 evaluation is scheduled for December 2026, which is the deadline the sequence below is sized against. The sequence is ordered by dependency rather than by date. Iterations 03b and 03c are independent of each other and may run in parallel; Iteration 04 depends on both, because the federated comparison is meaningless until the clients differ (03b) and incomplete until the second attack family exists (03c).

Table 9.2: Tentative FYP-2 timeline

Iteration	**Work**	**Exit condition**
03b	Extract `SyncStarvation`; sweep attack intensity downward; introduce differentiated per-zone traffic profiles	A genuine multiclass dataset exists, and the per-zone divergence of DEF-02 is measured rather than assumed
03c	Benign baseline run on the ring topology, 802.1CB sequence generation and recovery and configuration-level redundancy attack injection (done, Section 6.10); protocol-level redundancy attack; FRER feature family	The four behavioural criteria of Section 5.6.6 are met, and FRER residuals reach macro-F1 > 0.90 while the matching within-zone features remain at chance
04	Local-only, centralized-pooled and FedAvg arms on identical splits; zone attribution; detection latency; alert surface	SC-4 is answered either way: the federated arm is measured against both baselines on the same splits, and a null result is reported as a result
Final	Conformal calibration; monitoring-interference measurement; write-up	NFR-08 measured rather than carried over; final report and defence

Iterations 03b and 03c fall between this mid evaluation and the final FYP-1 evaluation in December 2026. Iteration 04 and the final items belong to FYP-2.

9.8 Expected Final Deliverables

Reproducible zonal TSN simulation with attack injection; labelled protocol-native feature dataset; trained detection model with federated and centralized training paths; comparative evaluation report covering detection, attribution, latency and federated behaviour; operator-facing alert surface; documented negative results and scope boundaries; complete source, configuration and handover package.

Chapter 10

Conclusion

This project addresses intrusion detection for the TSN Ethernet backbone of automotive zonal architectures, targeting attacks against TSN protocol features that degrade timing guarantees without altering packet content or traffic volume. The contribution is the use of protocol-native, schedule-derived signals as the detection feature space, trained federatively across zone controllers.

At the mid-report stage the data generation path is complete and reproducible: a four-zone TSN topology with gPTP synchronisation, three instrumented attack configurations, fifteen seeded runs and a labelled dataset of 4,740 feature windows. Delay-attack injection has been validated against its expected theoretical signature. The five-zone ring reference topology has since been built and used to simulate three further attack families: constant drift, asymmetric delay, and stream-redundancy attacks on an IEEE 802.1CB protected stream (Section 6.10). Each leaves a measurable signature in a protocol-mechanism metric (the gPTP rate ratio, the spatial pattern of offset shift, and the FRER elimination count) where the obvious metric is blind or ambiguous. These are single-run observations and are not yet extracted into the dataset.

The project's central premise has been tested ahead of schedule and holds. The test used a proof-of-concept centralized detector, built at reduced scale for the sole purpose of establishing feasibility. Features computed within a single zone detect the attack no better than a majority-class predictor; two cross-zone residual features reach 0.95–0.98 macro-F1, detecting two synchronisation intervals after onset with no missed attack windows in the best-performing arm. The gain is $+0.43$ to $+0.49$ macro-F1 and is insensitive to model choice, which is the evidence the contribution statement requires.

Two negative results bound the claim. Oscillator drift manipulation is not realisable as a runtime attack under the simulation framework, which reduced the committed attack set to two classes; a constant drift applied from the start of a run is hidden in the clock offset but visible in the servo's rate ratio on the ring, which makes it a candidate feature rather

than a result. More significantly, no fixed threshold on the offset residual separates attack from benign behaviour. The synchronisation convergence transient at run start produces residuals 2.7 times larger than the attack, and because those windows constitute roughly 11% of benign data, no percentile-based cut-off succeeds either. The benign class is a mixture of a convergence population and a steady-state population with the attack lying between them in magnitude, which is what makes a learned detector necessary rather than merely convenient, and what makes the second residual feature, weak on its own, the one that separates the two benign populations.

The remaining work is the detection model itself, which the proof of concept anticipated only at reduced scale, together with the federated training pipeline, the comparative evaluation, and extraction of the ring-topology signals, including the FRER feature family, into the dataset. The plan sequenced a central decision gate, the comparison between protocol-native and generic traffic features, ahead of federated infrastructure, so that the project's primary claim was tested before further work was built upon it; that gate has been passed.

Appendix A

Proposal Action Register: Supporting Evidence

The action register is presented in full in Section 3.8, where each item of panel feedback is recorded against the action taken, the evidence for it, and its current status. It is not duplicated here. This appendix holds the supporting material for that register.

Appendix B

Detailed Requirements

The functional requirements FR-01 to FR-12 and the non-functional requirements NFR-01 to NFR-08 are specified in Chapter 4, and traced to design elements, implementation artefacts and test cases in Table 4.4. The traceability matrix is the authoritative record; this appendix expands individual requirements where the one-line form in Chapter 4 is insufficient.

Appendix C

Detailed Test Cases

Two bodies of testing exist. The manual test cases TC-01 to TC-09 are specified in Section 7.3 and their results reported in Section 7.4. In addition, an automated suite runs on every push through continuous integration (Figure 8.1).

The automated suite contains fourteen tests in four groups. Three cover the windowing arithmetic, including an exact check that a ten-second run with two-second windows at one-second stride yields nine windows, since an off-by-one there would alter every row count reported in this document. Three cover the labelling rule, including the property that during an attack on one zone the other zones remain labelled benign; this is the test that encodes the correction recorded as DEF-01. Five cover the feature contract and the split: that no identifier column can reach a model, that the within-zone and residual feature sets are disjoint, and that no simulation run appears in both the training and test halves. Three are dataset invariants that execute only when `features.csv` is present, and so test the extracted dataset itself rather than a fixture: that attack rows are confined to the target zone, that the cross-zone residual is not identically zero, and that the victim zone rotates across runs.

Appendix D

Repository and Contribution Evidence

The repository holds the simulation sources, the extraction and training pipeline, the automated test suite and this report. Continuous integration evidence is reported in Section 8.5 and shown in Figure 8.1.

Repository. https://github.com/AdeenaZaib/VROOM

Per-member commit counts

Output of `git shortlog -sne`:

```
     4  AdeenaZaib <adeenazaib42@gmail.com>
     3  khaulabilqees <khbilq16@gmail.com>
```

Commit log

Output of `git log -pretty='%h %an %ad %s' -date=short`:

```
f009ac6  khaulabilqees  2026-10-13  update
31cb803  khaulabilqees  2026-10-13  update
2168990  AdeenaZaib     2026-10-13  add CI tests for feature extraction
                                     and training pipeline
856beba  AdeenaZaib     2026-10-13  adding the feature extraction stuff
ed4b0f6  AdeenaZaib     2026-10-13  added gitHub actions test workflow
8b765f6  khaulabilqees  2026-10-13  topology in omnett
0506e63  AdeenaZaib     2026-09-08  poc
```

The mapping from these commits to project milestones is given in Table 8.1, and the per-member attribution in Section 8.4.

Note on commit granularity. The history is coarse: seven commits, of which two carry the message `update`. This reflects the repository having been introduced partway through Iteration 1 rather than at its start, so earlier work was committed in bulk once version control was in place. Commit practice for FYP-2 is to commit per logical change with a message naming the module, in the form `M3: confine attack labels to the target zone`, so that the milestone mapping above can be produced mechanically rather than reconstructed.

Appendix E

Additional Design Diagrams

The principal diagrams appear in the body of the report: the system architecture and its interface contracts, the domain model, the level-1 data flow diagram, the extraction activity diagram and the detector state machine in Chapter 5; the implemented tree topology and the clock-offset traces in Chapter 6; and the five-zone ring reference topology in Section 5.6. This appendix holds the supplementary diagrams that support rather than carry the argument.

Appendix F

Meeting and Progress Evidence

The team met the supervisor weekly, on Tuesdays, throughout the reporting period. The table below records the sequence of those meetings by theme rather than listing each occurrence separately, since the weekly cadence was uninterrupted.

Table F.1: Supervisor meetings and their outcomes

Period	**Focus**	**Outcome and where it is recorded**
Before 11 September 2026	Proposal preparation and defence rehearsal	Proposal defended on 11 September 2026
Weeks following the defence	Review of the panel's comments, item by item	Six feedback items recorded with the action taken against each, in the action register (Section 3.8)
Subsequent weeks	Strategy for the work after the defence: scope, attack classes, and the order in which to establish the central claim	The decision to test the cross-zone feature claim ahead of any federated infrastructure, so that a null result would be found early (Section 3.3); the scope position recorded in Section 1.6
Weeks before the mid evaluation	Mid-evaluation preparation: evidence to present and gaps to close	Iteration 1 results consolidated; remaining work sequenced in Chapter 9

Assessment schedule. The proposal was defended on 11 September 2026. The mid evaluation falls in the week of 13 October 2026. The final FYP-1 evaluation is scheduled for December 2026.

Bibliography

[1] M. Adil, D. Javeed, and P. Kumar, "Selective federated IDS framework with adaptation for large-scale segmented industrial networks," *ACM Transactions on Internet Technology*, vol. 26, no. 3, pp. 1–31, 2026.

[2] T. T. Huong, T. P. Bac, D. M. Long, T. D. Luong, N. M. Dan, L. A. Quang, L. T. Cong, B. D. Thang, and K. P. Tran, "Detecting cyberattacks using anomaly detection in industrial control systems: A federated learning approach," *Computers in Industry*, vol. 132, p. 103509, 2021.

[3] P. Ruzafa-Alcazar, P. Fernández Saura, E. Marmol-Campos, A. González-Vidal, J. L. Hernández-Ramos, J. Bernal Bernabé, and A. Skármeta, "Intrusion detection based on privacy-preserving federated learning for the industrial IoT," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1145–1154, 2021.

[4] J. Zhang, C. Luo, M. Carpenter, and G. Min, "Federated learning for distributed IIoT intrusion detection using transfer approaches," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 7, pp. 8159–8169, 2022.

[5] J. Mao, Z. Wei, B. Li, R. Zhang, and L. Song, "FedIn-NID: A federated learning framework for network intrusion detection in large-scale heterogeneous industrial IoT," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 9250–9262, 2025.

[6] T. Hu, Q. Chen, Y. Hu, S. Hou, H. Yan, P. Yi, and Z. Cui, "Efficient and privacy-preserving network intrusion detection based on federated learning in SDN-enabled IIoT network," *IEEE Internet of Things Journal*, vol. 12, no. 20, pp. 41 904–41 923, 2025.

[7] Y. Wang, J. Li, Z. Han, P. Cheng, and R. Kumar, "FedSTGCN: A novel federated spatiotemporal graph learning-based network intrusion detection method for the Internet of Things," *Frontiers of Information Technology and Electronic Engineering*, vol. 26, no. 7, pp. 1164–1179, 2025.

[8] F. Al Tfaily, Z. Ghalmane, M.-e.-A. Brahmia, H. Hazimeh, A. H. Jaber, and M. Zghal, "Graph-based federated learning approach for intrusion detection in IoT networks," *Scientific Reports*, vol. 15, no. 1, p. 41264, 2025.

[9] M. T. Adil, D. Javeed, and P. Kumar, "Federated and quantized-based intrusion detection for embedded on-board automotive networks," *IEEE Transactions on Intelligent Transportation Systems*, 2026.

[10] G. Digregorio, E. Cainazzo, S. Longari, M. Carminati, and S. Zanero, "Evaluating the impact of privacy-preserving federated learning on CAN intrusion detection," in *Proc. IEEE 99th Vehicular Technology Conference (VTC2024-Spring)*, 2024, arXiv:2506.04978.

[11] Y. Shahsavari, R. Nourmohammadi, S. Rouhani, and K. Zhang, "SoK: Federated learning for intrusion detection in vehicular networks," 2026, arXiv:2607.10914.

[12] C. Park and S. Park, "Performance evaluation of zone-based in-vehicle network architecture for autonomous vehicles," *Sensors*, vol. 23, no. 2, p. 669, 2023.

[13] P. Meyer, T. Häckel, T. Lübeck, F. Korf, and T. C. Schmidt, "A framework for the systematic assessment of anomaly detectors in time-sensitive automotive networks," in *Proc. IEEE Vehicular Networking Conference (VNC)*, 2024.

[14] P. Meyer, T. Häckel, F. Korf, and T. C. Schmidt, "Network anomaly detection in cars based on time-sensitive ingress control," in *Proc. IEEE 92nd Vehicular Technology Conference (VTC2020-Fall)*, 2020.

[15] P. Meyer, T. Häckel, S. Reider, F. Korf, and T. C. Schmidt, "Network anomaly detection in cars: A case for time-sensitive stream filtering and policing," *Computer Networks*, vol. 255, p. 110855, 2024.

[16] K. S. V. V. Appajosyula, "Ethernet-based IDS for zonal architecture," in *SAE Technical Paper Series*, no. 2024-28-0121, 2024.

[17] T. Häckel, P. Meyer, F. Korf, and T. C. Schmidt, "Secure time-sensitive software-defined networking in vehicles," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 1, pp. 35–51, 2022.

[18] T.-G. Sorescu, N. Melnik, R. Rocha del Castillo, and R. Khondoker, "Threat analysis of over-the-air updates in distributed, domain-based and zonal E/E architectures," *Electronics*, vol. 15, no. 12, p. 2500, 2026.

[19] Z. Zhou, J. Lee, M. S. Berger, S. Park, and Y. Yan, "Simulating TSN traffic scheduling and shaping for future automotive Ethernet," *Journal of Communications and Networks*, vol. 23, no. 1, pp. 53–62, 2021.

[20] F. Luo, B. Wang, Z. Fang, Z. Yang, and Y. Jiang, "Security analysis of the TSN backbone architecture and anomaly detection system design based on IEEE 802.1qci," *Security and Communication Networks*, vol. 2021, pp. 1–17, 2021.

[21] D. Ergenç, R. Schenderlein, and M. Fischer, "TSNZeek: An open-source intrusion detection system for IEEE 802.1 time-sensitive networking," in *Proc. IFIP Networking Conference*, 2023, arXiv:2303.11492.

[22] X. Feng, Y. Wang, J. Lin, W. Li, S. Zhan, Y. Liu, J. Zhang, and J. Wang, "RobustTSN: A framework for protecting time-sensitive networking against unexpected delays," in *Proc. IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, 2024.

[23] N. Sertbaş Bülbül and M. Fischer, "Preemptive DoS attacks on time sensitive networks," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2023.

[24] A. M. Alnajim and C.-C. Shen, "Scheduling threats and their defenses in time-sensitive networks," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, 2022.

[25] S. U. Yücel, Y. E. Işıkçı, M. E. Peker, M. Evcil, İ. C. Babir, and Y. Çevik, "Detecting gPTP timing attacks in avionic time-sensitive networks," in *Proc. Integrated Communications, Navigation and Surveillance Conference (ICNS)*, 2026.

[26] C. Samp, N. Sertbaş Bülbül, and M. Fischer, "On the impact of gPTP time synchronization attacks on time-sensitive networks," in *Proc. IFIP Networking Conference*, 2026.

[27] A. de Vos, A. Brighente, and M. Conti, "Threat sensitive networking: On the security of IEEE 802.1cb and (un)effectiveness of existing security solutions," in *Computer Security. ESORICS 2022 International Workshops*, ser. Lecture Notes in Computer Science, vol. 13785. Springer, 2023, pp. 67–80.

[28] M. Fotouhi, A. Buscemi, A. Boualouache, F. Jomrich, C. Koebel, and T. Engel, "Assessing the impact of attacks on an automotive Ethernet time synchronization testbed," in *Proc. IEEE Vehicular Networking Conference (VNC)*, 2023.

[29] D. Berardi, N. O. Tippenhauer, A. Melis, M. Prandini, and F. Callegati, "Time sensitive networking security: Issues of precision time protocol and its implementation," *Cybersecurity*, vol. 6, p. 8, 2023.

[30] R. Kakade, J. Chou, and S. Torcato, "Vulnerability analysis of time synchronization in automotive Ethernet," 2022, arXiv:2208.11878.

[31] INET Framework Project, “INET framework for OMNeT++: TSN modules (TsnDevice, TsnSwitch, TsnClock),” 2024, version 4.7.0. [Online]. Available: https://inet.omnetpp.org/